

School of Computer Sciences

CAT403 Computing Infrastructure Major Project

System Requirements and Design Report

CogniShield: Large-Language-Model (LLM) Driven Campus Network Automation System

Lim Miin Ning

163133

Supervisor: Dr Vaithegy A/P Doraisamy

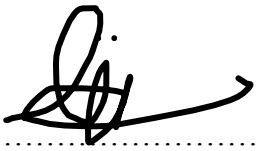
Examiner: Associate Professor Dr Chong Yung Wey

Academic Session

2025/2026

DECLARATION

"I declare that the following is my own work and does not contain any unacknowledged work from any other sources. This report was undertaken to fulfill the requirements of the Undergraduate Major Project for the Bachelor of Science in Computer Science (Honors) program at Universiti Sains Malaysia".

Signature : 

Name : ……..LIM MIIN NING..……

Date : ……10-DECEMBER-2025……

ABSTRAK

Pengurusan rangkaian dalam organisasi besar atau kampus selalunya rumit dan memakan masa kerana kebanyakan konfigurasi masih dilakukan secara manual. Walaupun terdapat alat automasi sedia ada, masih wujud jurang antara apa yang ingin dicapai oleh pengendali rangkaian dan apa yang perlu dikonfigurasi pada peringkat peranti. Lesen perisian proprietari dan kos yang tinggi turut mengehadkan akses bagi pelajar, perniagaan kecil, dan penyelidik. Projek ini mencadangkan CogniShield, sebuah sistem automasi rangkaian kampus yang dipacu AI dan berasaskan niat (intent-based), yang menggunakan model bahasa besar (LLM) untuk memahami arahan dalam bahasa semula jadi dan menukarnya kepada skrip konfigurasi automatik. Sistem ini juga boleh menjana respons masa nyata untuk isu rangkaian melalui automasi adaptif, menjadikannya lebih fleksibel dan pintar. CogniShield akan dibangunkan sebagai platform sumber terbuka dan neutral vendor bagi membolehkan universiti, perusahaan kecil, dan institusi penyelidikan mengguna serta bereksperimen dengan automasi rangkaian tanpa perlu membayar yuran lesen yang tinggi.

ABSTRACT

Network management in large organizations or campuses is often complicated and time-consuming because most configurations are still done manually. Even with existing automation tools, there is still a gap between what network operators want to achieve and what needs to be configured at the device level. Proprietary software licenses and high costs also limit accessibility for students, small businesses, and researchers. This project proposes CogniShield, an intent-based, AI-driven campus network automation system that uses large language models (LLMs) to understand natural language commands and translate them into automated configuration scripts. It can also generate real-time responses for network incidents and performance issues through adaptive automation, making the system more flexible and intelligent. CogniShield will be developed as an open-source and vendor-neutral platform to allow universities, small enterprises, and research institutions to adopt and experiment with network automation without paying high license fees.

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to everyone who supported me throughout the completion of this Final Year Project (FYP) entitled CogniShield: Large-Language Driven Campus Network Automation System. First and foremost, I am deeply indebted to my Supervisor, Dr. Vaithegy A/P Doraisamy, for her invaluable guidance, continuous encouragement, and insightful feedback throughout the entire project period. Her expertise was instrumental in shaping the direction and success of this research. A special thank you goes to my Examiner, Associate Professor Dr. Chong Yung Wey, for taking the time to evaluate my work and provide critical assessment. Finally, I would like to thank my family and friends for their unwavering support and motivation. This project would not have been possible without their patience and encouragement.

TABLE OF CONTENTS

DECLARATION 1

ABSTRAK 2

ABSTRACT 3

ACKNOWLEDGEMENTS 4

TABLE OF CONTENTS 5

LIST OF TABLES 7

LIST OF FIGURES 8

LIST OF ABBREVIATIONS AND SYMBOLS 9

Chapter 1 – Introduction 11

1.1 Background 11

1.2 Problem Statements 14

1.3 Motivation 16

1.4 System Objectives 17

1.5 Proposed Solutions 18

1.6 Benefits / Impact / Significance of Project 22

1.7 Uniqueness of Proposed Solution 24

1.8 Summary 25

1.9 Organization of the Report 26

Chapter 2 – Background and Related Work 28

2.1 Status of Project Development and Related Studies 28

2.2 SMEs Use Campus-Style Networks 29

2.4 Large-Language Models and Retrieval-Augmented Generation in Network Automation 36

2.5 Summary of the Background 38

Chapter 3 – System Requirements and Design 39

3.1 Project Scope, System Capabilities, and Limitations 39

3.1.1 Project Scope 39

3.1.2 System Capabilities — Module Description 40

3.1.3 System Limitations....46
3.2 Project Management....48
3.2.1 Work Breakdown Structure (WBS)....48
3.2.2 Gantt Chart....56
3.2.3 SWOT Analysis....58
3.3 Development Methodology — Waterfall SDLC....60
3.3.1 Planning....61
3.3.2 Analysis....63
3.3.2.1 Functional Requirement Analysis - Use Case Diagram....66
3.3.2.2 Functional Requirement - Use Case List....70
Module 1: LLM Playbook Generation....70
Module 2: Network Topology Visualization....81
Module 3: Playbook Validation & Execution....84
Module 4: Alert & Automated Remediation....93
3.3.2.3 Non-Functional Requirements....97
3.3.3 Design....99
3.3.4 Implementation....107
3.4 Technology Deployed and Hardware....115
3.4.1 Languages, Frameworks and Core Libraries....115
3.4.2 Hardware and Deployment Considerations....116
3.4.3 Security, Privacy and Safe-By-Design Choices....117
3.5 SDG Alignment....118
3.6 Conclusion & Future Works....119
Architecture and Implementation Guidelines....119
Testing Strategy and Hardware Recommendations....119
Future Works and Project Progression....120
References....121

LIST OF TABLES

Table 1: Comparison between CogniShield and Similar Existing Platforms……...32

Table 2: WBS table for Network Topology Visualization Module…….…….46

Table 3: WBS table for LLM Playbook Generation Module…....……..48

Table 4: WBS table for Playbook Validation & Execution Module…….50

Table 5: WBS table for Alert & Automated Remediation Module…..……52

LIST OF FIGURES

Figure 1: Three-tier Hierarchical Model……..11

Figure 2: Network Management Zone Connected to Core Layer Switches…….12

Figure 3: Integration of CogniShield to Campus Network in High-Level Architecture View……..21

Figure 4: Network Topology Visualization Module Diagram……..66

Figure 5: LLM Playbook Generation Module Diagram……...67

Figure 6: Playbook Validation & Execution Module Diagram……68

Figure 7: Alert & Automated Remediation Module Diagram…….69

Figure 8: Overall Workflow of CogniShield……..100

Figure 9: Intent Submission Sequence Diagram……103

Figure 10 Approval, Validation and Execution Sequence Diagram……..104

Figure 11: Alert and Automated Remediation Sequence Diagram……...105

Figure 12: Low-level CogniShield Architecture Diagram..…….…….108

LIST OF ABBREVIATIONS AND SYMBOLS

API - Application Programming Interface - A set of routines, protocols, and tools for building software and allowing communication between components

ACL - Access Control List - A list defining permissions attached to objects or resources

CLI - Command Line Interface - Text-based interface to interact with software or devices

CPU - Central Processing Unit - Primary processing unit of a computer or server

DB - Database - Organized collection of structured information or data

FYP - Final Year Project - Academic project undertaken by a university student in their final year

GUI - Graphical User Interface - Visual interface allowing users to interact with software via graphical elements

HTTP - HyperText Transport Protocol - Protocol for transferring hypertext requests and information on the web

JSON - JavaScript Object Notation - Lightweight data-interchange format used for structured data

LLM - Large Language Model - Machine learning model capable of understanding and generating human-like text

MVP - Minimum Viable Product - Simplified version of a system with core features to demonstrate functionality

NETCONF - Network Configuration Protocol - Network management protocol used for installing, manipulating, and deleting device configurations

REST - Representational State Transfer - Architectural style for designing networked applications

RAG - Retrieval-Augmented Generation - Process of using external context to improve the outputs of a language model

RBAC - Role-Based Access Control - Access control method that assigns permissions to users based on roles

SDN - Software Defined Networking - Networking architecture separating control plane from data plane to improve network programmability

SLA - Service-Level Agreement - Contractual agreement specifying performance and availability expectations

SQL - Structured Query Language - Language used for managing and querying relational databases

TLS - Transport Layer Security - Cryptographic protocol that provides secure communications over a network

UAT - User Acceptance Testing - Final phase of testing to ensure system meets user requirements

URL - Uniform Resource Locator - Reference to a web resource specifying its location

VM - Virtual Machine - Software emulation of a physical computer system

WBS - Work Breakdown Structure - Hierarchical decomposition of project tasks and deliverables

YAML - YAML Ain’t Markup Language - Human-readable data serialization standard often used for configuration files

Chapter 1 – Introduction

1.1 Background

Campus network, or also known as the campus area network, is a proprietary local area network (LAN) or a set of multiple interconnected LANs owned by a specific organization such as university, corporate or hospital (Contributor, 2025). In general, the campus network could span different geographical areas, from linking multiple university faculties in close distance to connecting headquarter and branch offices which are geographically dispersed. According to Upravnik (2022)**,** a campus network is typically designed using a three-tier hierarchical model, which involves the core layer, aggregation or distribution layer and finally the access layer for the purpose of scalability, reliability and security. Each layer comes with its own specialized functions, for instance, the core layer is used to provide high-speed routing as well as connect to other parts of the network like internet, network management zone, data center, demilitarized zone, and other campus branches. The aggregation layer, which connects the core layer to the access layer, is used to aggregate traffic from the access layer, perform the routing and filtering between different network segments, as well as enforce QOS and security policies. Lastly, the access layer, being the layer closest to the end-user, provides wired and wireless connection to terminals, like desktops, phones, wireless access points, and IoT devices (*Access, Distribution, and Core Layers Explained*, 2025).

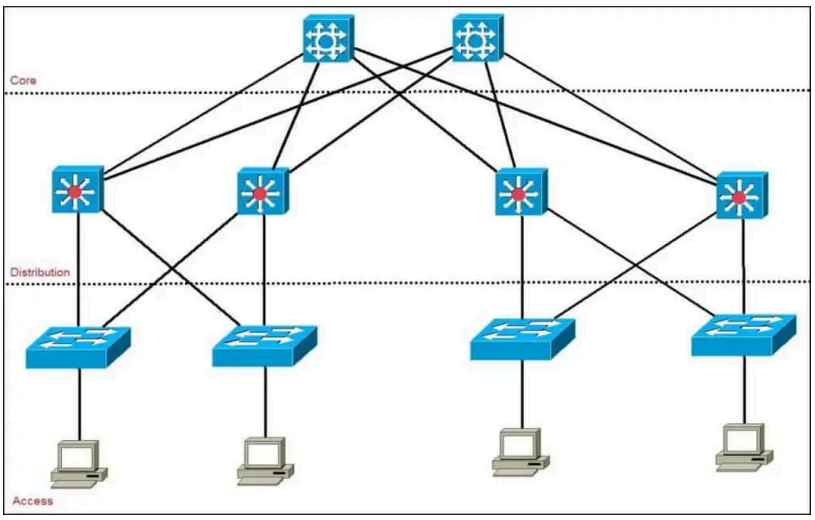


Figure 1: Three-tier Hierarchical Model (Upravnik, 2022)

With many devices and systems joining in the network, the campus networks have grown in scale and complexity, causing the network administrators to have a hard time to manage these devices (Huawei Technologies Co., Ltd., 2025). To work this out, the Network Management Zone is connected to the core layer switches, giving the administrators a central view of the whole network topology as well as allowing them to perform centralized network device administration and monitoring (Huawei Technologies Co., Ltd., n.d.). Nowadays, many vendors have also provided their management platform solution to support centralized network administration, for instance, Cisco DNA center, Juniper Mist, Huawei NCE, and Aruba Central.

However, these platforms fall short in respective different aspects. For example, most proprietary ecosystems are closed, others lacking multi-vendor interoperability, and almost all of them are too expensive to implement due to high-priced licence (*Cisco DNA Center Reviews & Product Details*, n.d.). Not only that, there also exists an obvious gap between operator's intention and the actual network commands in today's existing open-source and proprietary vendor platform. For example, an operator may want to isolate a department's network, but to do that, they need to manually write multiple vendor-specific configurations or navigate through different vendor tools. This process could require the operator to have vast knowledge of different vendor commands, not to mention it could possibly lead to errors, inconsistencies, and delays. Although some automation tools exist, many are expensive and only support specific vendors.

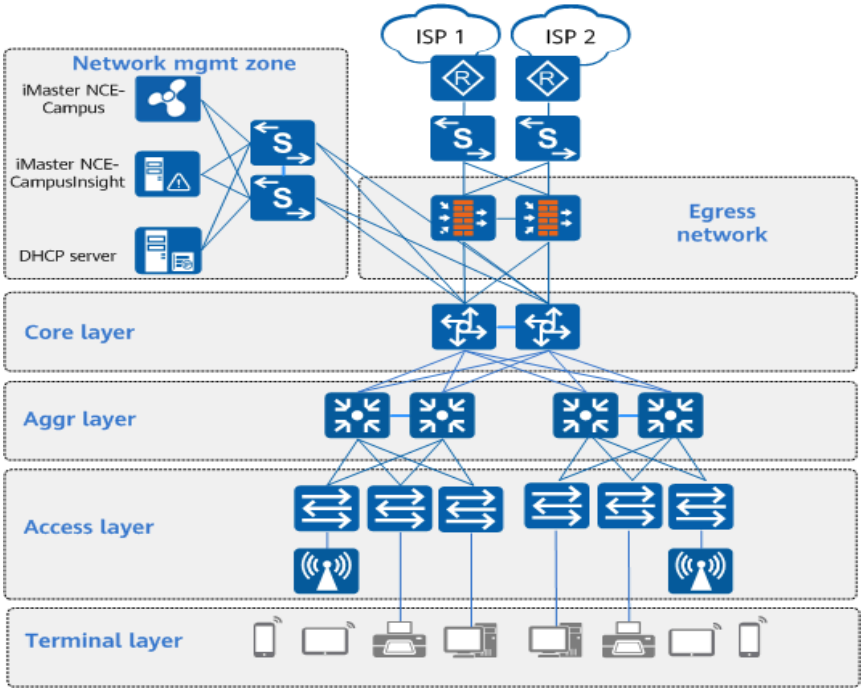


Figure 2: Network Management Zone Connected to Core Layer Switches (Huawei Technologies Co., Ltd., n.d.)

This project proposes **CogniShield**, an intent-based, AI-driven campus network automation system that uses large language models (LLMs) to understand natural language commands and translate them into automated configuration scripts. CogniShield will be developed as an open-source and vendor-neutral platform to make it easier for universities, small enterprises, and research institutions to adopt and experiment with network automation without paying high license fees.

1.2 Problem Statements

In today's world, there are numerous complex network automation systems used in campus or enterprise networks, where most of which are owned by well-known companies such as Cisco, Juniper, Aruba, and Huawei. Despite the fact that these platforms are quite robust, for instance Cisco DNA Center, Juniper Mist, and Huawei iMaster NCE, they all have one thing in common, which is they are not widely accessible or open to the public (*Open Networking Alternative to Cisco, Juniper, Arista, Nokia*, 2025). The primary reason here is due to their proprietary licensing system. In this context, licenses, software subscriptions, and hardware are all extremely expensive, making them tough for small and medium-sized businesses, startups, and even certain institutions to purchase (*Cisco DNA Center Reviews & Product Details*, n.d.). What is more distressing is that there are no other viable open-source alternatives for the network community. Such circumstances present a significant challenge for individuals who cannot afford or wish to construct their own configurable network automation or zero-trust system without relying on a provider. As a result, the organization will have to invest more time, money, and labor in the traditional manual configuration and monitoring of its network. Moreover, the proprietary licensing also affects students, researchers, and network hobbyists by restraining them from investigating or learning how these network automation tools work. And because the majority of the resources and underlying architecture are restricted and not released publicly, it is difficult to conduct community reviews or collaborative development with the public (*Free and Open Source Software in Education | Research Starters | EBSCO Research*, n.d.). As a result, innovation faces stagnation and open, low-budget, vendor-neutral network automation systems struggle to come into existence.

Aside from that, another issue in the current network field is that, while there are many automation tools or management platforms available, the majority of them lack a fully functional LLM feature that can automatically convert network engineers' high-level intentions into real-world device configurations. For example, if a network administrator wishes to segregate one department's network, he must still manually write commands or navigate through many interfaces such as Command Line Interface (CLI), Application Programming Interface (API), or Graphical User Interface (GUI) of various vendor devices to implement firewall rules (*5 Intent-Based Networking Vendors*, n.d.). This manual method takes a long time and requires extensive technical knowledge of each vendor's system and

configuration syntaxes. As the number of devices increases and the network infrastructure gets increasingly complex, such manual processes would be less inefficient and induce human errors such as configuration drift or inconsistency. Ultimately, this would significantly hinder the overall system's flexibility and scalability (“Transforming Network Configuration Management: Challenges and Solutions,” 2022).

1.3 Motivation

It was my experience doing an internship at Huawei Malaysia that inspired me to start this project. Back then, I was given a lot of repetitive network tasks. I realized most of these tasks followed a predefined routine and could therefore be automated. Since then, I have written many automation scripts to facilitate the workflow, which saved a lot of time and energy. Later on, what truly sparked my interest in doing this automation project, however, was getting to see how their internal network management platform, which is Huawei iMaster NCE, works and looks like. From the tool, I got a glimpse on how the network configurations and administration which I have learned in the CST337 course instructed by Dr Chong Yung Wey could also be easily done and automated in a real-world environment. On top of that, I was also fortunate enough to be given the opportunity to study for and take the Huawei Certified ICT Professional (HCIP) Network Automation Developer course and exam, where I have learned vast amount of knowledge on automation technology, concepts, and tools, which further exposed me to the possibilities and future of AI and automation in networking.

Given this internship exposure and certification opportunity, I decided to explore this technology more deeply. That is when I realized the technology is largely locked within proprietary vendor ecosystems and not accessible to everyone. Surprisingly, there is still no existing open-source system that allows students like me or universities in general to get hands-on experience. That is when I came up with an idea, to create an open-source network system particularly for the campus network. I decided to start this project after realizing how much time and effort network administrators could save when provisioning the network infrastructure. This tool could be beneficial and of good use to those small enterprises like start-ups, small institutions or students who could not afford the pricey commercial platform and would like to customize the tools to their needs. I want to build something that makes automation easier, faster, and open for everyone with no more licence restrictions. With CogniShield, I aim to bring AI-powered reasoning into the network while keeping it simple and affordable for educational and research purposes.

1.4 System Objectives

1. To develop an open-source, vendor-neutral, zero-trust network orchestration platform that promotes accessibility and innovation.

2. To implement an LLM-powered, intent-based network automation system that can translate high-level commands into vendor-specific configurations.

3. To develop adaptive automation that can respond to network incidents and performance issues in real time (e.g, link failures, device misconfigurations, traffic congestion) while keeping human-in-the-loop for critical changes.

1.5 Proposed Solutions

In hope of solving issues faced in the problem statement, this project proposes a new concept called CogniShield, an intent-based network automation framework powered by the Large-Language Model (LLM). It is neither a simple network management tool, nor a closed commercialized platform, but an AI-driven open-source automation system, aiming to provide a free community alternative that would also make the network as intelligent, flexible and reliable as the existing platforms. Ultimately, this network automation controller shall be connected to the campus network and placed at the network management zone as shown in Figure 3.

To begin with, the entire concept of CogniShield is based on open-system architecture, in which the whole system is built upon open-source components and does not require expensive software subscription or hardware components to run it. With that being said, the system could run and function seamlessly on top of any standard servers with the help of virtualization technology like Docker and Proxmox (*"What Is Docker?,"* 2024; Proxmox, n.d.). All these to ensure that small-to-middle enterprises, startups, and research institutes could also effortlessly build and use the system without needing to worry much about the hardware costs, licensing and vendor restrictions.

From the perspective of the architecture, the system is made up of multiple stable open-source components, such as Netbox for network device library management, Ansible for automation, and SDN controller for centralized control (Ansible Documentation — Ansible Community Documentation, n.d.). In terms of monitoring and visualization, CogniShield would use SNMP, Telemetry and Prometheus to perform monitoring, InfluxDB for time-series databases, and Grafana for visualization (*Overview | Prometheus*, n.d.; InfluxDB OSS V2 Documentation, n.d.; *About Grafana | Grafana Documentation*, n.d.). In this project, open architecture is highly emphasized with the goal of promoting the cost-saving, flexibility and interoperability, not to mention allowing the enterprises or users to develop and customize the system based on their environment and operational needs. For students, researchers, and community developers, the system serves more like a free experimental field, where they could explore, innovate and share their research together on this open platform, contributing their findings and promoting community-driven development.

Amongst all the components integrated, the Large-Language Model (LLM) is the heart and core of the CogniShield, which it intends to understand and implement the natural language command, thereby bridging the gap between human intent and the device configuration. For example, when the network administrator invokes an instruction like “Segmenting IoT devices from the main VLAN”, the system will automatically generate the corresponding automation scripts in the form of Ansible playbook. In the process of generating the automation scripts, real-time topology information from the SDN controller and Netbox inventory data will be referenced to ensure the remediation proposed is context-aware (*Software-Defined Networking (SDN) Definition - Open Networking Foundation*, 2020). This process lessens the complex steps when manually writing multi-vendor configuration scripts, minimizes the human error risk, as well as enabling operators without extensive vendor knowledge to be able to manage the complex network easily.

What is more about CogniShield is that it does not only translate the user intent into configuration commands, it can also serve as a “thinking assistant”, especially when it comes to facing unexpected or sudden network context changes. In many conventional network systems, responses to network events often depend heavily on predefined rules and playbooks to counter anticipated or happened incidents (Sharma, 2025). However, CogniShield tries to break this limit by being more perceptive and situationally adaptive. It does this by first receiving the information from SDN, SNMP, and telemetry to get the real-time data and understand the whole network context. Then, it would use the LLM reasoning to generate the appropriate solution. To picture this, if the system encounters sudden topology changes, link performance drops, device failures, or unusual shift in traffic patterns, it would learn the latest network topology, identify the affected points, and then generate or propose new configuration playbooks based on the contextual information to stabilise the network. In general, these remediation steps could involve redirecting traffic through another available path, adjusting the QoS of certain traffic or rebalancing load across multiple links. This feature allows the system to be more adaptive and self-learning, making it evolve in the dynamically changing network environment.

Another point worth to note is that regardless of how much automation works it could bring to the table, CogniShield is only meant to assist the engineer in fixing the network errors. At some points, it may automatically fix minor issues that have little to none effects on its own. But when it comes to critical recalibrations that could bring huge impacts to the overall

network, the configurations proposed still need to undergo manual reviews and approval from the engineers first before the actual implementation. Human intervention is still needed considering the LLM response may not always be fully correct, or worse, it could be exploited by prompt injection into giving malicious or unsafe configuration commands which will lead to security breaches (*Towards Trustworthy AI: Characterizing User-Reported Risks Across LLMs "In the Wild,"* n.d.). That being said, this system ultimately should only serve as an assistant tool to ease the configuration and remediation workflow rather than a fully autonomous AI network engineer that could act on its own.

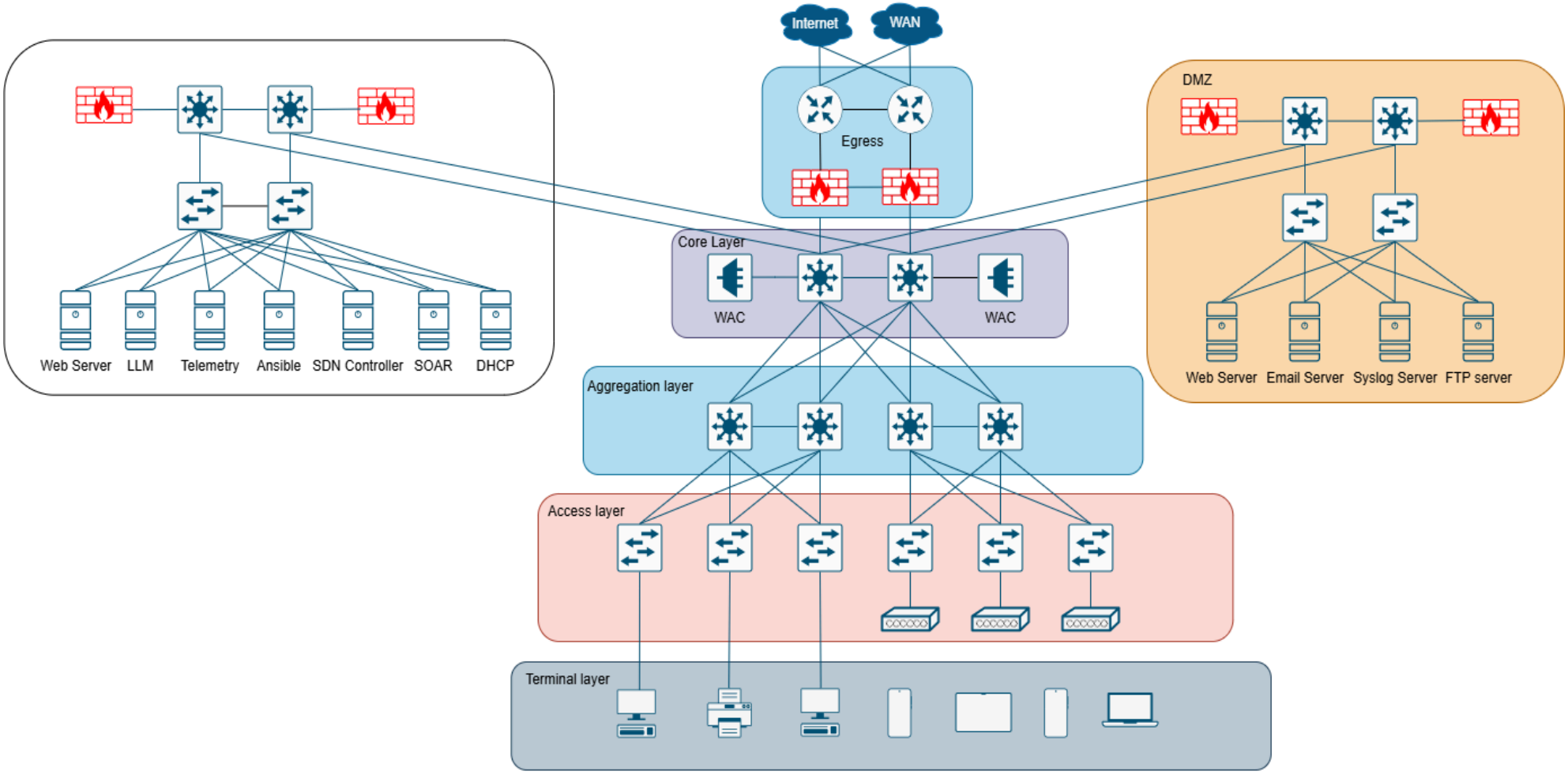


Figure 3: Integration of CogniShield to Campus Network in High-Level Architecture View

1.6 Benefits / Impact / Significance of Project

For Network Admins and IT Professionals:

- **Less Human Work and Error**: With repeated configuration efforts automated by CogniShield, administrators could spend more time on network planning instead of performing manual configuration. It also reduces human error and configuration drift with automatic conversion of human intent into scripts (*What Is Multidomain Intent-Based Networking?*, 2025)
- **Faster Network Incident Response**: The AI-powered monitoring and adaptive automation enables more efficient detection, investigation, and issue resolution for issues like network device or link failures. This could reduce human intervention at some points as well as minimize downtime as well as improve overall system resilience.
- **Cross-Vendor Flexibility**: Since CogniShield is open-source software that is vendor-independent, it is compatible with devices from various vendors without relying on any ecosystem. This assists organizations with mixed hardware environments in saving money as well as efforts.
- **Enhanced Network Visibility**: With interoperation with SDN controllers, NetBox, and monitoring software, administrators can see real-time device states as well as topology in one place, which simplifies device troubleshooting

For Educational and Research Organizations:

- **Open Experimentation and Learning**: Because CogniShield is open-source software, students as well as researchers can access the integrated framework, understand how the automation works behind, and further improve or customize it according to their needs without any licensing restrictions.
- **Hands-on Network Management Coursework**: The project can be used as a platform for research organizations to train students on existing network automation using a realistic experimental system.
- **Low-Cost Deployment**: The CogniShield can be installed on standard servers or software platforms such as Proxmox and Docker without proprietary hardware requirements nor software subscriptions that are costly.

For Small Organizations and Small Businesses:

- **Low Provisioning Cost**: Since CogniShield is open-source, small organizations could afford to use network automation without expensive licensing charges.
- **Simplified and Efficient Operations**: Automated daily repetitive configurations, software update, as well as monitoring. Overall, automation can help small organizations manage stable networks with minimal technical personnel.
- **Enhanced Network Posture**: The adaptive behavior of the system due to LLM and dynamic playbook is capable of spotting unusual or unexpected network patterns, identify affected components, and generate real-time configuration adjustments to stabilise the network. This helps maintain overall network performance, resilience, and reliability even during rapid or unpredictable changes in traffic or topology.

1.7 Uniqueness of Proposed Solution

When it comes to the uniqueness of the project, CogniShield, without a doubt, may not be the first to introduce the idea of intelligent centralized network automation. However, CogniShield still managed to set it apart from the rest of the competitors with some features and functionalities that others lack. Unlike the existing vendor platforms, CogniShield is free and open-source. No proprietary ecosystem and no licensing is required in order to use the system. Not only that, CogniShield is also considered as vendor-neutral seeing the fact that it does not need any software or hardware purchase and can run on top of standard bare-metal server using Proxmox and Docker. In short, these features make the system an ideal choice for students, SMEs, and research organizations due to its affordability and openness.

Beyond the perspective of the cost, CogniShield's LLM-oriented architecture also gives it another major edge. While some vendors like Cisco and Huawei iMaster NCE have just recently developed and integrated the LLM in their network user-intent translation module, CogniShield aims to deliver a fully LLM-integrated automation and orchestration system that would work across the entire network layer. In this context, the LLM would help generate dynamic playbooks for network configurations and operational orchestration. Such features would enable natural-language driven automation, perform tasks like dynamic AI-driven adjustments, and provide intelligent recommendations based on the user intent and the overall network context.

In short, CogniShield's uniqueness lies in its openness, affordability and complete ecosystem independence, forming a well-rounded solution that provides an advanced LLM automation platform while delivering a full range of features and functionalities some vendor platforms would often miss or lack. The following table shows a comprehensive comparison between CogniShield and the existing solutions in terms of the features and functionalities.

1.8 Summary

This chapter provides an overview of the project called CogniShield. It begins by explaining the background of campus networks and the difficulties in managing big network systems. As networks become more complicated and require more adaptive capabilities, it is observed that existing proprietary systems have inherent limitations and struggle to provide better automation and configuration support. With that being said, these challenges highlight the need for a free, open-source, intelligent, and user-friendly network automation system.

The chapter also talks about the main problems faced, such as the lack of open-source tools, limited use of AI for translating user intentions into configuration, and the system's inability to keep up with rapidly changing network conditions. The motivation for the project comes from the student's own experience during an internship, where he saw real-world network automation systems. This inspired the creation of a free, vendor-neutral, AI-powered platform.

The goals of the system are to solve these problems, including developing an open-source orchestration platform, using AI to translate user intentions, and supporting adaptive, real-time configuration adjustments. The proposed solution, CogniShield, was described in detail, highlighting its architecture, AI-driven features, and how it makes network management tasks easier. The chapter also explains the benefits and special qualities of the system, such as being user-friendly, adaptable, and suitable for schools, research, and small businesses.

In summary, this chapter sets the stage for the report by explaining the context, challenges, goals, and solution of the project, along with the reasons and importance of CogniShield. The next chapters will go into more detail about designing, building, testing, and evaluating the system.

1.9 Organization of the Report

From this chapter onwards, the report is structured to guide the reader through the detailed study and development of the CogniShield system. Following the current chapter, Chapter 2, **Background and Related Work**, provides the general context of network automation and intent-based management, highlighting the challenges and requirements that motivate this project. It presents the problem statements that the system aims to solve, explains the motivations for carrying out the study, and defines the system objectives. The proposed solutions are described in detail, including the system architecture, overall module structure, and the unique aspects of the platform that differentiate it from existing approaches. This chapter also reviews relevant past projects and literature to provide context, introduce key concepts, and explain terminology, ensuring that readers can understand the technical and operational foundations for the project.

Chapter 3, **System Requirements and Design**, focuses on translating the concepts and proposals from the background into actionable system specifications. The chapter begins by defining the project scope, system capabilities, and limitations, followed by project management planning including the Work Breakdown Structure (WBS), milestone timelines, and Gantt charts to illustrate scheduling and resource allocation. A SWOT analysis is presented to highlight the strengths, weaknesses, opportunities, and risks related to the project. The development methodology is explained in the context of the system's requirements and operational needs, showing why a structured approach such as the Waterfall SDLC is selected for safe and controlled implementation.

This chapter then elaborates the detailed system requirements, both functional and non-functional, and presents analysis using appropriate diagrams including UML use case diagrams, class diagrams, sequence diagrams, and process flowcharts. Each module of the system is described in terms of purpose, functionality, and interaction, with module diagrams and architecture diagrams supporting clarity. Where applicable, the sources of training and test data are explained to support LLM grounding and validation. The chapter also discusses the technology deployed, including hardware specifications, software tools, programming languages, frameworks, libraries, and operating systems used during development and deployment, ensuring reproducibility in the laboratory environment and providing guidance for future expansion.

Overall, the report is organized to progress logically from background and motivation, through detailed requirements and design, to implementation, testing, and operational considerations. Each subsequent chapter builds on the previous ones, allowing the reader to understand the system in a structured way and see how the design decisions and technical implementations are informed by the initial research and planning.

Chapter 2 – Background and Related Work

2.1 Status of Project Development and Related Studies

Campus networks are usually built using the three-tier hierarchical model consisting of core, aggregation (or distribution), and access layers, and this model remains widely used because it separates responsibilities clearly, supports scalability, and helps with reliability and security (Cisco Systems, 2008). The literature and vendor documents continue to treat the three-tier architecture as the canonical reference for campus design, which makes it an appropriate starting point for discussing automation and orchestration efforts (Cisco Systems, 2019). Over recent years, research and industry practice have moved towards intent-based networking, software-defined networking, and centralized automation, where high-level operational intentions such as “segment guest devices” or “isolate compromised endpoints” are translated automatically into device-specific commands (Cisco Systems, 2020).

Alongside this, there has been significant interest in integrating artificial intelligence into network operations, and many recent studies or prototypes explore how large language models (LLMs) and other AI techniques can help administrators by understanding human instructions and generating configuration scripts or recommended actions (Wang et al., 2024). The combination of intent-based approaches with LLMs, especially when grounded by retrieval or RAG techniques, appears promising because it allows models to reference actual network data before producing outputs (Lewis et al., 2020). Given these developments, CogniShield should be understood as an enhancement and continuation of existing research directions; it builds on intent-based and closed-loop automation ideas while explicitly integrating an LLM plus retrieval layer so that generated automation is context-aware and vendor-neutral (Cisco Systems, 2020; Wang et al., 2024).

2.2 SMEs Use Campus-Style Networks

While the term "campus network" is usually associated with very large institutions such as universities, hospitals, or corporate campuses, network design literature and real-world SME technology solutions show that small and medium enterprises (SMEs) are increasingly adopting similar layered, modular topologies (Network Computing, 2016; Contributor, 2025). Vendor design guidance and enterprise profiles explain how campus architectures can be scaled down for smaller organisations, preserving the access, aggregation or distribution, and core layering but adapting them for cost-sensitive and space-constrained environments (Cisco Systems, 2008; Access, distribution, and core layers explained, 2025). In practice, many vendors recommend a collapsed-core topology that merges core and aggregation functions to reduce device count and cost while keeping logical separation and manageability (Network Computing, 2016).

For example, Cisco's Small Enterprise Design Profile recommends layer-2 and layer-3 access options with a collapsed-core topology to achieve resiliency and modularity without over-provisioning hardware (Cisco Systems, 2008). Huawei's CloudCampus and SME guides provide reference architectures for small- and medium-sized sites and suggest either a two-tier design (access plus collapsed core) for moderate SMEs, or a full three-layer design for larger SME offices (Huawei Technologies Co., Ltd., 2025). Vendor solution documents for SME offices often list core switches, access switches, wireless access points, and gateway or firewall functions, which indicates a layered rather than flat topology in their recommended deployments (Huawei Technologies Co., Ltd., 2025).

Academic and engineering research also models multi-layer LAN topologies and centralised management for small business networks, describing segmentation, access-layer switches, controllers, and forwarding devices consistent with campus-style infrastructures (Wang et al., 2024; Transforming Network Configuration Management, 2022). Recent papers and preprints discuss scalable hierarchical layouts, VLAN segmentation, and redundancy trade-offs that apply to SMEs as well (Ismail et al., 2025; Wang et al., 2024).

Operationally, SMEs share the same basic motivations as larger organisations: segmentation and security to separate device classes, scalability so the network can grow without a full redesign, and resiliency that limits fault domains (OECD, 2021; U.S. International Trade Administration, 2023). Although many SMEs use manual configuration or lower-cost tools

instead of high-end orchestration platforms, a logical layered structure still makes it easier to apply policies, quality-of-service rules, and segmentation even with simpler tooling (NetBox Project, n.d.; NetBrain, 2024).

Taken together, vendor reference designs, academic studies, and SME-focused solutions suggest that campus-style network principles are relevant for SMEs and are often implemented in simplified or collapsed forms that preserve the key benefits of modularity, manageability, and resilience (Cisco Systems, 2019; Huawei Technologies Co., Ltd., 2025). At the same time, most public examples are vendor reference architectures or research prototypes rather than independent production case studies, so more empirical survey and procurement data would help quantify how many SMEs run full three-layer campus deployments (Network Computing, 2016; Transforming Network Configuration Management, 2022).

In conclusion, while the concept of a campus network may seem tailored to large-scale enterprises, the evidence from vendor solutions, design profiles, and academic research strongly supports the conclusion that SMEs are using campus-style network architectures (Cisco Systems, 2008; Huawei Technologies Co., Ltd., 2025; OECD, 2021; Wang et al., 2024). These enterprises may choose simplified or collapsed versions, but the layered, modular principles remain intact. Operational drivers such as segmentation, scalability, management, and resiliency motivate this adoption, and although some empirical gaps remain, the weight of design guidance and solution architectures indicates that campus-style networks are very much part of the SME networking landscape (Cisco Systems, 2019; Huawei Technologies Co., Ltd., 2025; U.S. International Trade Administration, 2023).

2.3 Existing Systems and Critical Comparison

Existing solutions for network automation can be roughly grouped into two categories, which are commercial integrated platforms and modular open-source toolchains. Commercial vendor platforms such as Cisco DNA Center, Juniper Mist/Marvis, Huawei iMaster NCE, and NetBrain offer polished, end-to-end solutions that combine telemetry analytics, assurance, intent interpretation, automation, and lifecycle management (Cisco Systems, 2023). These products are strong in terms of deep vendor-specific integration, enterprise-grade support, and mature analytics (Cisco Systems, 2023; Juniper Networks, 2022). However, they commonly suffer from high licensing costs, vendor lock-in, limited transparency, and restricted multi-vendor interoperability, which makes them impractical for many students, researchers, and small organisations (Cisco Systems, 2023; G2 reviews summary, 2022).

Cisco DNA Center is one of the most recognized enterprise-grade network automation platforms, offering a comprehensive suite for intent-based networking, automation, and assurance. The platform allows administrators to define high-level business intent and then automatically translate it into device-level configurations, supported by telemetry and analytics that continuously monitor the network for compliance and anomalies (Cisco, n.d.; Campus Technology, 2025). While Cisco DNA Center has recently begun integrating AI-first initiatives to provide dynamic insights and automated recommendations, these capabilities are still emerging, and much of the automation historically relies on static rules or models. The platform primarily focuses on network orchestration, offering limited security integration, and it requires proprietary Cisco hardware or virtual appliances to operate. Additionally, the cost and licensing model make it inaccessible for small enterprises or research and educational use. Although DNA Center is robust and highly integrated for Cisco environments, its vendor-centric approach and high entry cost limit its flexibility and broader accessibility, making it less suitable for the kinds of open, vendor-neutral deployment that CogniShield aims to provide (Cisco, 2023).

Juniper Mist, particularly through its Marvis AI module, represents a significant step forward in AI-driven intent-based networking. Marvis employs machine learning and natural language interfaces to allow administrators to interact with the network conversationally, query status, and receive automated remediation suggestions (Juniper Networks, n.d.; Conry-Murray, 2025). Unlike DNA Center, Juniper Mist has more mature AI-native features

and can provide predictive insights and automated network remediation in response to detected issues. However, the platform is still partially proprietary and requires Juniper hardware and cloud infrastructure for full functionality, limiting flexibility in mixed-vendor environments. The closed ecosystem and licensing restrictions also constrain its use for research and educational purposes. While the AI capabilities and intent-based features are relatively advanced, Juniper Mist remains less accessible and affordable than an open-source solution like CogniShield, which combines AI-driven automation with vendor neutrality and full local deployment options (Juniper Networks, 2022).

Huawei iMaster NCE is another enterprise-oriented platform that integrates network automation, assurance, and orchestration for multi-domain campus and enterprise networks (Huawei Enterprise, n.d.). It provides a digital twin architecture that supports closed-loop, lifecycle management of network configurations and operational monitoring, and includes predictive analytics and intent-based features. However, iMaster NCE is largely optimized for Huawei devices, which can limit interoperability with other vendors, and it requires dedicated Huawei appliances or virtualized instances to function properly. Its automation features are primarily predictive rather than generative, and licensing costs are high, making it impractical for small enterprises or educational use. Although Huawei NCE offers robust network management and partial AI-driven capabilities, the proprietary and hardware-dependent design constrains its openness and flexibility, particularly when compared to CogniShield's fully open-source, containerized approach that enables deployment in any environment without licensing restrictions (Huawei Enterprise, n.d.).

On the other hand, open-source stacks assembled from components like NetBox for inventory, Ansible for automation, Prometheus or InfluxDB for telemetry, and Grafana for visualization provide transparency, flexibility, and affordability,, making them particularly suitable for learning and experimentation (NetBox Project, n.d.; Ansible Community, n.d.; Prometheus, n.d.; Grafana, n.d.). However, these components are less integrated by default and do not offer a unified, natural-language intent layer, requiring considerable effort to combine and often relying on static rules or playbooks for automation (NetBox Project, n.d.; Ansible Community, n.d.). Many existing systems follow rule-based orchestration, policy engines, telemetry analytics, and, in some cases, machine learning for predictive insights or automated suggestions (Cisco Systems, 2020; Juniper Networks, 2022). While these approaches are deterministic and auditable, they are brittle in the face of new or unexpected

scenarios and require continuous maintenance (Ismail et al., 2025), and machine-learning-based analytics risk producing unsafe recommendations if not grounded in accurate, current network models (Wang et al., 2024).

When comparing these open-source toolchains and commercial platforms such as Cisco DNA Center, Juniper Mist, and Huawei iMaster NCE with CogniShield, it becomes clear that the proposed system integrates the best aspects of both worlds while addressing their limitations. Unlike commercial platforms, CogniShield is fully open-source and vendor-neutral, deployable on standard containerized or virtualized servers without expensive hardware or licensing, making it ideal for students, SMEs, and research institutions. While DNA Center and Huawei NCE offer partial AI-driven automation and Juniper Mist provides conversational AI insights, CogniShield combines a large language model (LLM) with retrieval-augmented generation (RAG) to enable context-aware, natural-language intent automation and dynamic remediation in real time. CogniShield focuses on network automation, providing closed-loop state remediation and AI-guided recommendations while retaining human oversight to ensure safe operations. In essence, CogniShield bridges the gap between fragmented open-source stacks and high-cost proprietary platforms, offering an integrated, intelligent, adaptive, and accessible automation framework that is flexible, scalable, and safe for diverse network environments (Lewis et al., 2020; Wang et al., 2024

Feature / Functionality	CogniShield	Cisco DNA Center [C1]	Juniper Mist [J1]	Huawei iMaster NCE [H1]	NetBrain [N1]
Open-Source / Vendor- neutral	✅	⚠️ (Proprietary, Cisco-centric with some limited 3rd-party support)	⚠️ (Proprietary, partially vendor- neutral)	⚠️ (Proprietary, supports multi-vendor but optimized for Huawei)	⚠️ (Proprietary, Supports multi-vendor)
AI-Driven Dynamic Remediation	✅	⚠️ (Emerging "AI-first" initiatives; Historically uses model/rule)	✅ (AI-native AIOps; AI insights & suggest automated remediation)	⚠️ (Predictive, intent, automation engine but non- generative)	✅ (AI-powered, intent-based automation)
Intent-Based Natural Language Automation	✅	⚠️ (Emerging Intent-Based LLM support)	✅ Intent-Based LLM support)	⚠️ (Emerging intent/LLM support)	⚠️ (Intent-based no-code automation, not full natural-language)
Closed-Loop State Remediation	✅	⚠️ (Partial automation, manual approvals)	⚠️ (Partial automation, manual approvals)	✅ (Full life-cycle, digital twin closed-loop)	⚠️ (Drift detection, partial automation only)

Hardware Purchase	✅ (Runs on virtualized servers like Proxmox)	❌ (Requires Cisco appliance or virtual)	❌ (Requires Juniper appliance and cloud SaaS)	❌ (Require Huawei appliance, VM and Cloud)	⚠️ (On-premise deployment)
Cost & Accessibility	✅ (Free and open-source, ideal for students/SMEs)	❌ (Very high, enterprise-only)	❌ (High)	❌ (High)	❌ (Medium)
AI Prompt & Automation Recommendation	✅ (LLM + natural language)	⚠️ (Historically Lack LLM support, recently adding AI-Assistant)	⚠️ (Some conversational/intent connectors)	⚠️ (LLM "One Master" concept)	⚠️(Partial AI-assistant and insights support; limited LLM orchestration.)
Educational & Research Use	✅	❌ (License restricted)	❌ (License restricted)	❌ (License restricted)	⚠️ (Limited by pricing)

Table 1: Comparison between CogniShield and Similar Existing Platforms

2.4 Large-Language Models and Retrieval-Augmented Generation in Network Automation

Large Language Models (LLMs) are powerful artificial intelligence systems trained to understand and generate human-like text, including complex sequences such as software code, automation scripts, and step-by-step instructions. Their strength lies in their ability to interpret natural language queries and produce structured outputs that previously required manual human effort. In software development, LLMs have been integrated into tools like Visual Studio Code, where they enhance productivity by analyzing the cursor position, understanding the surrounding code context, and even cross-referencing other files within the project. This allows the model to generate up to 100% complete code that is not only syntactically correct but also relevant and functionally consistent across modules. This "code-as-you-type" assistance dramatically accelerates development, reduces errors, and allows developers to focus on higher-level logic rather than repetitive tasks.

The same concept can be extended to network automation. Network engineers often perform tasks that are essentially "network-as-code": configuring devices, writing automation playbooks, managing firewall rules, or creating response sequences for network incidents. By leveraging LLMs, these tasks can be described in plain language for example, "Update the routing configuration for all switches in the finance department to enforce VLAN segmentation" and the model can generate the corresponding commands, scripts or Ansible playbooks automatically. This approach not only speeds up repetitive work but also lowers the barrier for engineers and students who may not be familiar with every device-specific command syntax. LLMs can therefore transform natural language instructions into actionable network operations, bridging the gap between human intent and machine execution.

Despite their generative strengths, LLMs have limitations. They do not inherently possess knowledge of the current state of a specific network or its real-time configurations. For instance, an LLM might generate commands based on general networking principles but could conflict with the current topology, device settings, or policy constraints. In other words, while LLMs are excellent at reasoning and generating content, they cannot be relied upon as authoritative sources of environment-specific facts without access to current, structured data (Bender et al., 2021; Huang et al., 2023).

This limitation is addressed by **Retrieval-Augmented Generation (RAG)**, which combines the generative abilities of LLMs with the precision of authoritative data sources. In a RAG framework, the system first retrieves relevant information, such as device inventories from NetBox, telemetry snapshots, network topology from SDN controllers, configuration templates, or historical playbooks and then feeds this context to the LLM. By grounding the model in actual network data, RAG ensures that the generated commands, scripts, or playbooks are accurate and aligned with the current environment (Lewis et al., 2020; Lewis et al., 2020, NeurIPS paper). For example, a network engineer could prompt the system to "Generate a remediation playbook for all routers showing high CPU usage," and the RAG-enabled LLM would first retrieve real-time telemetry, device configurations, and previous successful remediation steps before producing a reliable, executable sequence.

While RAG significantly improves reliability, it introduces additional engineering complexity. Maintaining retrieval pipelines, vector stores, and ensuring the freshness of indexed data requires careful design. Moreover, even RAG outputs can occasionally hallucinate or misinterpret information, making verification and human oversight essential. Practical safeguards include dry-run simulations, digital-twin validations, and manual approval gates, which prevent unsafe commands from being executed on live systems (Wang et al., 2024; Farquhar et al., 2024).

CogniShield integrates this LLM+RAG approach into network automation, producing context-aware configuration snippets, dynamic playbooks, and remediation steps while ensuring that verification layers prevent potentially harmful actions. A practical illustration could involve a campus network experiencing intermittent outages across multiple labs: CogniShield can automatically retrieve device states, topology, and historical incident data, then generate a remediation script that engineers can review before execution. This approach combines the speed, intelligence, and natural-language capabilities of LLMs with the reliability and safety of data-grounded automation, making advanced network orchestration both accessible and trustworthy for students, SMEs, and researchers (Lewis et al., 2020; Wang et al., 2024).

In essence, LLMs provide the reasoning and generative power, RAG ensures factual grounding, and together they enable "network-as-code" automation that is dynamic, context-aware, and safe, bridging the gap between static rule-based approaches and fully automated intelligent orchestration.

2.5 Summary of the Background

In summary, the existing literature and commercial landscape show an industry trend towards intent-based, telemetry-driven automation and an emerging role for LLMs; however, most powerful solutions today are proprietary and expensive, which limits their adoption by students, researchers and SMEs (Cisco Systems, 2023; OECD, 2021). Commercial platforms are strong in polish and support but are closed and costly, whereas open-source components are flexible and affordable but require integration work and typically lack an intelligent, natural-language intent layer (NetBox Project, n.d.; Ansible Community, n.d.). Static security playbooks remain common but are insufficient for novel threats, and purely generative LLM outputs need grounding through retrieval and careful validation to avoid hallucination and unsafe actions (Bender et al., 2021; Lewis et al., 2020; Huang et al., 2023). Given these observations, CogniShield positions itself as an open-source, vendor-neutral continuation and enhancement of current research and product directions by combining a digital-twin inventory, established automation engines like Ansible and SDN control for execution, robust telemetry and visualization for situational awareness, and an LLM+RAG layer to translate human intent into context-aware automation (NetBox Project, n.d.; Ansible Community, n.d.; Prometheus, n.d.; Grafana, n.d.; Lewis et al., 2020; Wang et al., 2024). CogniShield emphasizes safety through verification and human approval for critical actions, while still enabling dynamic, adaptive playbook generation for lower-risk remediation (Ismail et al., 2025). The following chapters will provide detailed architecture, design choices, implementation methods and evaluation plans that explain how each component is implemented and how safe closed-loop automation is achieved with CogniShield.

Chapter 3 – System Requirements and Design

3.1 Project Scope, System Capabilities, and Limitations

3.1.1 Project Scope

CogniShield is an open-source, vendor-neutral, intent-based campus network automation platform developed as a student project and targeted at universities, small and medium enterprises, and research labs. The system's MVP (minimum viable product) scope concentrates on the network configuration lifecycle from human intent to validated execution: a web-based intent composer that accepts plain-English instructions; a RAG-lite context gatherer that collects NetBox inventory snapshots, recent telemetry slices and a topology snapshot; an LLM adaptor that generates candidate Ansible/CLI playbooks and concise human-readable justifications; a static validator and an Ansible check-mode dry-run facility that produce diffs and risk scores; a controlled approval and execution orchestrator that applies changes to a lab (EVE-NG) or permitted devices; a rollback facility that uses pre-change snapshots; a topology visualization module showing planned vs. live maps and diffs; and an alert/incident detector that suggests remediation playbooks for operator review. The project intentionally limits scope to networking concerns (configuration, orchestration, observability) and excludes application lifecycle orchestration, full production high-availability, and on-premise operation of very large foundation LLMs (GPU-heavy inference). The design emphasizes demonstrability in a student lab: all components should be runnable in Docker containers and tested against scripted EVE-NG topologies and a NetBox inventory so the work is reproducible in an academic environment.

3.1.2 System Capabilities — Module Description

Network Topology Visualization Module

The Network Topology Visualization Module provides a complete environment for representing, comparing and analysing both the planned and live states of the network. It serves as the system's visual intelligence layer, enabling users to understand network structure, track deviations and initiate corrective workflows through a clear, evidence-centred interface.

The module maintains two complementary topology perspectives. The **planned topology layer** is sourced directly from NetBox, which acts as the authoritative system of record for the intended network design. This planned view reflects expected node–link relationships, device roles, interface assignments and VLAN mappings based on the inventory and documentation stored in NetBox. By relying on an established source of truth rather than user-uploaded diagrams, the planned layer ensures that design expectations remain consistent, traceable and aligned with formal network documentation.

The **live topology layer** retrieves real-time discovery and state information from LLDP/CDP snapshots, SDN controller northbound APIs, or EVE-NG interrogations. This layer displays the actual operational topology, including interface states, active neighbour relationships and basic telemetry indicators. Together, these two complementary views allow operators to examine both intended and real layouts within the same visual environment.

A key capability of this module is the **planned-versus-live topology differencing engine**. The engine highlights deviations such as missing or unexpected links, inconsistent device roles, VLAN mismatches, stale records or topology drift. Each diff element is interactive: selecting an inconsistency opens the corresponding NetBox entry and can immediately generate a pre-filled remediation request within the playbook workflow. This allows users to move seamlessly from identifying discrepancies to initiating automated corrective actions.

The module supports **telemetry overlays**, enabling real-time metrics such as interface operational state, utilization or recent error counts to be displayed directly on the topology nodes and edges. Additionally, a **time-window playback feature** allows users to inspect historical states of the network for troubleshooting, demonstration or audit purposes.

To ensure accessibility within a university project environment, the user interface is intentionally lightweight and browser-based, typically employing libraries such as Cytoscape.js or Vis.js. This ensures smooth performance even for moderately sized campus or lab topologies while maintaining clarity, responsiveness and usability.

Knowledge-Grounded LLM Playbook Generation Module

The Knowledge-Grounded LLM Playbook Generation Module provides a unified, evidence-driven pipeline that transforms natural-language operator objectives into structured, context-aware configuration playbooks. It functions as the core intelligence layer of the system, ensuring that generated outputs are grounded in real network conditions, traceable to the user's intent, and suitable for downstream validation and execution.

The workflow begins when an operator articulates a network objective using natural language. Each submission is stored as a structured **intent record** enriched with metadata such as creator identity, timestamp, scope, and priority. These records support common operational workflows including browsing past entries, cloning intents for reuse, editing drafts prior to confirmation, and cancelling requests that are no longer required. By capturing operational goals in a formalised and auditable structure, the system preserves both historical traceability and repeatability.

Once an intent is captured, the module's **context-gathering engine** automatically assembles authoritative and up-to-date information relevant to the requested change. This includes device metadata from NetBox such as platform type, operating system version, interface structure and VLAN assignments along with topological relationships retrieved from LLDP/CDP snapshots or SDN controller state. A short telemetry window from Prometheus or InfluxDB is also included, capturing recent interface statistics, alarms, or performance anomalies. Instead of retrieving complete datasets, the engine intelligently selects the most relevant "top-k" items, such as affected devices, their immediate neighbours, recent error conditions and short-term performance indicators. These elements are packaged into a compact JSON evidence bundle that accompanies the intent, ensuring that subsequent reasoning is supported by verified network facts rather than high-level assumptions.

The **LLM playbook generator** consumes both the operator intent and the curated evidence bundle. Using structured prompt templates, the system requests the model to produce one or

more candidate Ansible playbooks or CLI configuration sequences, each accompanied by a concise justification describing the rationale, referenced evidence and expected impact. Every generated result is stored as a **Playbook Record**, which includes the YAML/CLI output, provenance data, checksums, timestamps and any descriptive notes. Operators may regenerate variants with refined constraints, compare candidates side-by-side, or select a preferred version to progress into the validation pipeline. For student deployment, the module supports both hosted LLM APIs and lightweight local models, enabling flexible experimentation without hardware overhead.

Overall, the module provides a disciplined, knowledge-grounded transformation pathway that converts ambiguous operator goals into reproducible and auditable configuration artefacts. By combining structured intent capture, authoritative context packaging and controlled LLM-based generation, the system ensures that automated playbooks are not only syntactically correct but also operationally meaningful within the real network environment.

Playbook Validation & Execution Module

The Playbook Validation & Execution Module provides a comprehensive safety and governance framework that ensures all configuration playbooks generated by the system are thoroughly analysed, verified, safely executed, and recoverable in the event of unexpected outcomes. It functions as the central enforcement layer responsible for maintaining operational correctness, reducing execution risk, and preserving a complete audit trail for every network change.

The module begins with a rigorous **static and policy-based validation process** applied to every incoming playbook. Static validation includes syntax checks for YAML and CLI formats, vendor-specific linting, module compatibility verification, and structural correctness assessments. Complementing these checks, policy validation enforces governance rules that safeguard critical network assets. Examples include restricting high-impact operations on core devices, validating the use of reserved VLAN ranges, and detecting IP conflicts through cross-checks against authoritative NetBox inventory data. The module also produces a calculated risk score that considers factors such as the number of affected devices, their roles in the topology, and outcomes of similar historical executions. All validation findings are delivered in structured reports that clearly differentiate between errors, warnings, and

suggested corrections. Based on these results, playbooks may be rejected, revised by the operator, or allowed to proceed to the simulation phase.

For safe pre-deployment assessment, the module offers a **Dry-Run & Simulation subsystem** that performs what-if analysis using Ansible's check mode and diff capabilities. Simulations run either in sandboxed environments, such as EVE-NG instances or isolated lab devices and capture before-and-after snapshots of all affected configuration fragments. These snapshots are used to generate human-readable diffs that detail the exact changes the playbook would introduce. In cases where certain Ansible modules do not fully support check mode, the subsystem highlights these tasks explicitly, ensuring that operators are aware of operations requiring special caution. All simulation outputs are permanently attached to the corresponding intent record, enabling thorough auditing and peer review.

Once validation and simulation results have been reviewed, the **Approval Engine** manages authorization for live execution. Low-risk operations may require only a single endorsement, whereas higher-risk or broader-scope tasks can be configured to follow a staged multi-approval workflow. After approval, the playbook is executed through a controlled orchestration system, typically using Ansible Runner with limited concurrency and enforced sequencing to reduce operational risk. During execution, real-time logs are streamed to the user interface, and the system conducts post-application verification checks to confirm that the intended state such as expected interface status, VLAN modifications, or routing adjustments has been correctly realised.

To ensure resilience in operational workflows, the module includes a dedicated **Rollback subsystem**. Before any configuration changes are applied, it captures minimal, targeted pre-change snapshots covering precisely the configuration elements that may be modified. If execution deviates from expectations whether due to device errors, verification failures, or operator intervention, the subsystem automatically generates and applies rollback playbooks to restore the previous network state. After restoration, it performs verification checks to ensure that the rollback has been successful. This capability is designed to support the scope and change patterns typical of controlled lab environments, while acknowledging that highly complex rollback scenarios remain outside the MVP scope.

Through its layered design which comprises validation, simulation, authorization, execution, and recovery, the Playbook Validation & Execution Module ensures that automation-driven network changes remain safe, predictable, transparent, and fully auditable.

Alert & Automated Remediation Module

The Alert & Automated Remediation Module provides a comprehensive framework for network monitoring, incident detection, context-aware analysis, and automated corrective actions. It is responsible for continuously observing the operational state of the network, identifying anomalies, and facilitating safe and well-structured remediation workflows. This module is designed to ensure that issues are detected early, assessed intelligently, and addressed in a controlled manner, thereby maintaining network stability and supporting operational resilience.

The module continuously monitors diverse telemetry sources, including interface statistics, CPU and memory utilization, error counters, routing stability, and device health metrics. It identifies abnormal conditions such as link failures, persistent errors, performance degradation, and unusual traffic patterns. Upon detecting an incident, the system automatically generates an incident record and attaches a comprehensive context bundle. This bundle includes detailed device metadata, network topology relationships, recent telemetry snapshots, and historical event data. By grounding all decisions in authoritative network data, the module ensures that subsequent actions are well-structured and traceable.

Once an incident is captured, the module can trigger the LLM-based remediation engine to generate candidate corrective playbooks. Each proposed remediation undergoes rigorous validation, simulation, and approval processes, identical to those applied to manually submitted intents or playbooks. The system leverages safety controls, risk scoring, and simulation previews to guarantee that automated recommendations are reliable and aligned with operational policies. Unsupported or high-risk actions are clearly flagged for operator review, maintaining transparency and accountability in automated workflows.

For routine or low-risk scenarios, administrators can define policy rules that allow the module to automatically apply remediation actions without requiring explicit approval. Examples include clearing minor interface errors, restarting non-critical services, or adjusting non-core configuration parameters. However, in typical deployments, particularly within academic or

student lab environments, manual approval remains the default, highlighting the importance of safety, governance, and operator oversight.

The module maintains a detailed record of every incident, including its detection, context, remediation proposals, validation outcomes, execution logs, and any rollback actions. These records support post-incident review, performance tuning, and continuous improvement of the automated remediation policies. Historical analysis of incidents also allows administrators to identify recurring issues, optimize remediation strategies, and refine alert thresholds.

Additionally, the module supports advanced telemetry overlays and visualization tools, enabling operators to observe patterns and trends over time. This historical insight enhances the understanding of network behavior under different load conditions, assists in training simulations, and facilitates academic demonstrations. By closing the operational loop from detection to reasoning to remediation, the Alert & Automated Remediation Module transforms raw telemetry data into actionable insights while enforcing strict safety and governance principles.

In summary, this module ensures proactive network management by detecting problems, reasoning about their causes using context-aware evidence, generating corrective actions, and, when permitted by policy, executing them automatically. Its comprehensive logging, verification, and governance mechanisms make it a robust, reliable, and auditable component of the CogniShield platform.

3.1.3 System Limitations

CogniShield is constructed with explicit technical and operational limitations that should be stated clearly so evaluators understand the project bounds.

LLM limitations and hallucination risk: One significant limitation arises from the use of large language models is LLM hallucination. In other words, LLM models are inherently probabilistic and may generate plausible yet incorrect or unsafe commands. To mitigate this risk, CogniShield treats all model outputs as suggestions rather than authoritative instructions. Every generated playbook undergoes static validation, dry-run previews, and operator approval before any high-impact changes are applied. Additionally, the platform records the RAG evidence associated with each generation, allowing operators to inspect the context and reasoning used by the model. Despite these safeguards, it remains possible for false or suboptimal recommendations to occur, necessitating careful human oversight.

Hardware and model constraints: Hardware and model constraints also define the system's operational limits. Running large on-premise foundation models typically requires GPUs and specialized expertise, which are beyond the typical resources available in a university student project. Consequently, the minimum viable product (MVP) supports either hosted LLM APIs or small local models that can run efficiently on CPU hardware. Full on-premise inference for very large models is intentionally excluded. Similarly, the computing resources provided by EVE-NG lab hosts and student machines impose memory and CPU limitations, constraining the size and complexity of test topologies. To ensure reproducibility and manageability, the design emphasizes minimal, well-scoped topologies suitable for demonstration and grading.

Partial Ansible check-mode coverage: Partial coverage of Ansible check-mode functionality is another limitation. This is because not all Ansible modules or vendor APIs support safe, side-effect-free simulations. As a result, dry-run outputs may be incomplete for certain operations, and tasks that cannot be safely simulated are flagged for manual review or require controlled application within the lab environment. The project documents which modules are safe for dry-run and which require operator caution to ensure safe testing practices.

Telemetry and discovery coverage: The effectiveness of the LLM and automation pipeline is dependent on the freshness and completeness of telemetry and discovery data.

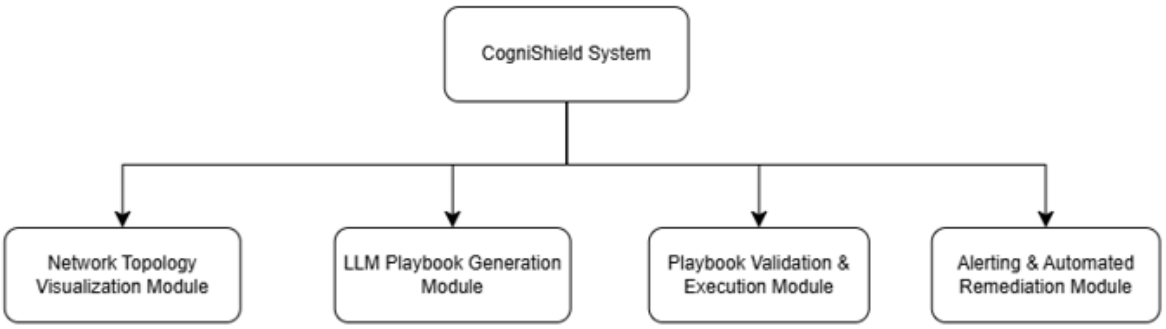
Environments where inventory data is outdated, LLDP or SDN snapshots are missing, or telemetry sampling intervals are long may reduce the contextual accuracy of generated playbooks. Within the MVP, NetBox snapshots and telemetry scrapes are scheduled frequently to maintain reliability in the test harness, but gaps may exist in real networks, affecting automation confidence.

Security and credential management: Security and credential management also impose additional operational boundaries in this project. CogniShield handles sensitive device credentials, and in the student implementation, these are managed using Ansible Vault or a simple secrets store along with strict procedural controls. Full enterprise-grade security features, such as secret rotation, Hardware Security Module integration, and identity federation, are beyond the MVP scope but are considered for future enhancement.

Scope and scale limits: Finally, the project's scope and scale are explicitly limited to campus or small-to-medium enterprise networks and student lab demonstrations. Extremely large carrier networks, hyperscale deployments, multi-tenant cloud fabrics, or complex cross-domain orchestration are outside the project boundaries. Similarly, the validator rule set and the catalog of playbook templates are initially small and curated. While broad vendor and operational coverage could be added in the future, the current design focuses on a manageable, demonstrable subset of network automation use cases.

3.2 Project Management

3.2.1 Work Breakdown Structure (WBS)



Network Topology Visualization Module

Activity	Date	Duration (Days)
1. Project Planning		
a. Define project scope	01/10/2025	3
b. Develop WBS and build schedule for visualization module	04/10/2025	4
c. Risk Planning	08/10/2025	4
2. Analysis Task		
a. Gather and analyze Netbox/EVE-NG data structure	12/10/2025	5
b. Identify key telemetry & network topology attributes	17/10/2025	4
c. Define and prioritize visualization requirements (planned/live/diff)	21/10/2025	4
d. Model network visualization flow using UML	25/10/2025	4
3. Design Task		

a. Design database or cache for topology snapshots	29/10/2025	4
b. Design GUI layouts for topology visualization	02/11/2025	5
c. Design diff computation logic & overlay telemetry	07/11/2025	4
4. Build Task		
a. Implement topology data import/export	11/11/2025	5
b. Implement planned, live, and diff visualization	16/11/2025	6
c. Overlay basic telemetry on network map	22/11/2025	5
d. Perform unit testing: visualization logic	27/11/2025	4
e. Perform integration test: GUI + backend + DB	01/12/2025	4
f. Perform system and acceptance test	05/12/2025	6
Note: The use cases that will be developed in this module: ● UC15 - Visualize planned network topology ● UC16 - Visualize real-time network topology ● UC17 - Compare planned vs real-time topology (topology diff) ● UC18 - Drift detection & topology-change notification		

Table 2: WBS table for Network Topology Visualization Module

LLM Playbook Generation Module

Activity	Date	Duration (Days)
1. Project Planning		
a. Define project scope	06/10/2025	3
b. Develop WBS and build schedule and then plan the work	09/10/2025	4
c. Risk Planning	13/10/2025	4
2. Analysis Task		
a. Gather and analyze intent requirements	17/10/2025	5
b. Review and analyze existing automation solutions	22/10/2025	4
c. Define and prioritize intent-related requirements	26/10/2025	4
d. Model intent flow using UML (use case + activity diagram)	30/10/2025	4
3. Design Task		
a. Design intent database scheme	03/11/2025	4
b. Design GUI layouts for intent composer	07/11/2025	5
c. Design API endpoints for creating/editing intents	12/11/2025	4
4. Build Task		
a. Build intent database entities	16/11/2025	4
b. Build GUI for intent submission and list	20/11/2025	5
c. Perform unit testing: intent creation/editing	25/11/2025	4
d. Perform integration test: GUI + backend + DB	29/11/2025	4

e. Perform system and acceptance test	03/12/2025	6
Note: The use cases that will be developed in this module: ● UC01 - Submit intent ● UC02 - View intent history ● UC03 - Edit intent ● UC04 - Cancel intent ● UC05 - Duplicate intent ● UC06 - Retrieve network inventory context ● UC07 - Retrieve telemetry context ● UC08 - Retrieve forwarding and SDN state ● UC09 - Construct RAG bundle ● UC10 - Update vector embeddings ● UC11 - Generate playbook ● UC12 - Provide justification ● UC13 - Regenerate playbook ● UC14 - Compare candidate playbooks		

Table 3: WBS table for LLM Playbook Generation Module

Playbook Validation & Execution Module

Activity	Date	Duration (Days)
1. Project Planning		
d. Define project scope	04/12/2025	3
e. Develop WBS and build schedule and then plan the work	07/12/2025	4
f. Risk Planning (Execution/Rollback Safety)	11/12/2025	3
2. Analysis Task		
e. Gather and analyze validation rules (syntax, vendor, conflicts)	14/12/2025	5
f. Define approval workflows and roles	19/12/2025	4
g. Analyze rollback scenarios and triggers	23/12/2025	4
h. Model end-to-end flow using UML	27/12/2025	4
3. Design Task		
d. Design static validator engine and error reporting	31/12/2025	4
e. Design approval interface and workflow logic	04/01/2026	5
f. Design execution engine with rollback	09/01/2026	5
4. Build Task		
f. Implement static validator (YAML/CLI + rules)	14/01/2026	6
g. Implement approval UI and workflow	20/01/2026	5
h. Implement execution engine (Ansible runner + logging)	25/01/2026	5
i. Implement rollback mechanism (snapshot-based)	30/01/2026	5

j. Perform unit testing: validator + execution + rollback	04/02/2026	4
k. Perform integration test: full workflow	08/02/2026	4
l. Perform system and acceptance test	12/02/2026	5
Note: The use cases that will be developed in this module: ● UC19 - Perform static validation ● UC20 - Perform risk scoring ● UC21 - Policy decision for auto-apply ● UC22 - Detect policy violation ● UC23 - Provide validator report ● UC24 - Perform dry-run simulation ● UC25 - Show configuration diffs ● UC26 - Capture simulation logs ● UC27 - Store dry-run artifacts ● UC28 - Approve playbook ● UC29 - Reject or request changes ● UC30 - Execute playbook (apply) ● UC31 - Post-execution basic verification ● UC32 - Save pre-change snapshot ● UC33 - Execute rollback		

Table 4: WBS table for Playbook Validation & Execution Module

Alert & Automated Remediation Module

Activity	Date	Duration (Days)
1. Project Planning		
g. Define project scope	15/02/2026	3
h. Develop WBS and build schedule	18/02/2026	4
i. Risk Planning (False positives, unintended remediation)	22/02/2026	3
2. Analysis Task		
i. Gather and analyze telemetry and alert types	25/02/2026	5
j. Define auto-remediation rules	02/03/2026	4
k. Identify GUI notification requirements	06/03/2026	4
l. Model alert → remediation workflow using UML	10/03/2026	4
3. Design Task		
g. Design alert collection/storage	14/03/2026	4
h. Design auto-remediation executor	18/03/2026	5
i. Design GUI notification	23/03/2026	4
4. Build Task		
m. Implement alert collector and notifier	27/03/2026	5
n. Implement auto-remediation executor	01/04/2026	6
o. Integrate full workflow	07/04/2026	5
p. Perform unit testing	12/04/2026	4
q. Perform integration test	16/04/2026	4

r. Perform system and acceptance test	20/04/2026	5
Note: The use cases that will be developed in this module: ● UC34 - Monitor basic thresholds ● UC35 - Notify operator ● UC36 - Generate remediation suggestion ● UC37 - Execute auto-remediation ● UC38 - Request manual review		

Table 5: WBS table for Alert & Automated Remediation Module

3.2.2 Gantt Chart



WEEKLY GANTT CHART TEMPLATE

CogniShield: LLM Driven Campus Network Automation System	Lim Miin Ning	1-Oct-2025	20-Apr-2026
PROJECT NAME	OWNER	START DATE	END DATE

Project Description

CogniShield is a student-developed platform for safe and automated network configuration, monitoring, and remediation in lab and small-to-medium enterprise environments. It features modules for network topology visualization, LLM-assisted playbook generation, validation and execution, and alert-driven automated remediation. The project follows a Waterfall schedule to ensure systematic planning, design, implementation, and testing with clear documentation and auditability.

Remarks

Task ID	Task Name	Start Week	End Week	Resources	WK01	WK02	WK03	WK04	WK05	WK06	WK07	WK08	WK09	WK10	WK11	WK12	WK13	WK14	WK15	WK16	WK17	WK18	WK19	WK20	WK21	WK22	WK23	WK24	WK25	WK26	WK27	WK28	WK29	WK30
Module 1:	**Network Topology Visualization**																																	
01	***Project Planning***																																	
01.01	Define module scope	WK02	WK03			1	1																											
01.02	Build WBS & risk notes	WK04	WK05					1	1																									
02	***Analysis***																																	
02.01	Analyse NetBox/EVE-NG data	WK06	WK07							1	1																							
02.02	Identify telemetry/topology attributes	WK08	WK09									1	1																					
03	***Design***																																	
03.01	Design snapshot/DB structure	WK10	WK11											1	1																			
03.02	GUI layout + diff logic	WK12	WK13													1	1																	
04	***Implementation***																																	
04.01	Implement data import/export	WK14	WK15															1	1															
04.02	Build visualization (Planned/Live/Diff)	WK16	WK17																	1	1													
04.03	Telemetry Overlay	WK18	WK19																			1	1											
05	***Testing***																																	
05.01	Unit Testing	WK20	WK21																					1	1									
05.02	System & Acceptance Testing	WK22	WK23																							1	1							
Module 2:	**LLM Playbook Generation**																																	
06	***Project Planning***																																	
06.01	Define module scope	WK06	WK07							1	1																							
06.02	Build micro-WBS & risk notes	WK08	WK09									1	1																					

07	***Analysis***		
07.01	Analyse intent workflow	WK10	WK11
07.02	Define requirement + UML	WK12	WK13
08	***Design***		
08.01	Intent DB Design	WK14	WK15
08.02	GUI + API Design	WK16	WK17
09	***Implementation***		
09.01	Build DB + GUI	WK18	WK19
09.02	Integration tasks	WK20	WK21
10	***Testing***		
10.01	Unit Testing	WK22	WK23
10.02	System & Acceptance Testing	WK24	WK25
Module 3:	**Playbook Validation & Execution**		
11	***Project Planning***		
11.01	Define module scope	WK10	WK11
11.02	Build WBS & risk notes	WK12	WK13
12	***Analysis***		
12.01	Analyse validation rules & workflows	WK14	WK15
12.02	Define requirements + UML	WK15	WK16
13	***Design***		
13.01	Validator + Execution pipeline design	WK17	WK18
13.02	Approval workflow + rollback design	WK19	WK20
14	***Implementation***		
14.01	Build validator, approval UI	WK21	WK22
14.02	Integrate execution engine + rollback	WK23	WK24
15	***Testing***		
15.01	Unit Testing	WK25	WK26
15.02	System & Acceptance Testing	WK27	WK28
Module 4:	**Alert & Automated Remediation**		
16	***Project Planning***		
16.01	Define module scope	WK14	WK15
16.02	Build micro-WBS & risk notes	WK16	WK17
17	***Analysis***		
17.01	Analyse alert sources + telemetry	WK18	WK19
17.02	Define requirements + UML	WK20	WK21
18	***Design***		
18.01	Alert engine + storage design	WK22	WK23
18.02	Auto-remediation + notification design	WK24	WK25
19	***Implementation***		
19.01	Build alert listener + remediation engine	WK26	WK27
20	***Testing***		
20.01	Unit Testing	WK28	WK29
20.02	System & Acceptance Testing	WK29	WK30

3.2.3 SWOT Analysis

Strengths

The CogniShield system offers several key advantages as an open-source solution. It is designed to be free and open-source, significantly promoting accessibility and innovation in network automation. It achieves vendor-neutrality and cross-vendor flexibility by running on standard containerized or virtualized servers such as Docker or Proxmox without requiring proprietary hardware or expensive software licensing. The system's central feature is its LLM-powered, intent-based automation, which translates high-level natural language commands directly into vendor-specific configuration scripts, speeding up processes and reducing human error. Furthermore, CogniShield provides AI-driven dynamic remediation, allowing it to generate context-aware, adaptive solutions and new playbooks in response to novel security events or abnormal traffic, moving beyond the static rule-sets of conventional SOAR systems. It integrates unified network and security orchestration, a feature often lacking in competing vendor platforms that focus primarily on one domain.

Weaknesses

The primary limitation of CogniShield stems from its reliance on Large Language Models (LLMs): the LLM response may not always be fully correct. Therefore, the system is designed to serve only as an assistant tool, and critical recalibrations require manual reviews and approvals from engineers before implementation to maintain operational safety. There is an inherent security risk that the LLM could be exploited by prompt injection to provide malicious or unsafe configuration commands, which necessitates the need for human oversight. Also, while aiming to be a unified platform, the core architecture is built upon the integration of multiple open-source components like Ansible, Netbox, and Prometheus, which requires careful design to ensure seamless operation and stability.

Opportunities

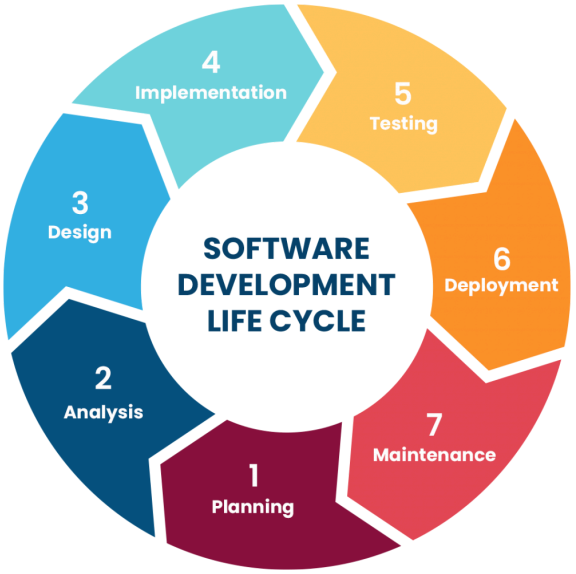
CogniShield is ideally positioned to address the market gap left by proprietary automation and security platforms, such as Cisco DNA Center, Juniper Mist, and Huawei iMaster NCE, which are often too expensive due to high licensing costs and are not widely accessible. Its affordability and openness make it the perfect platform for universities, researchers, and

small and medium enterprises (SMEs) to adopt network automation and security orchestration technologies, fostering open experimentation and learning. The system aligns with the emerging industry trend of integrating LLMs for intent-based networking, leveraging retrieval-augmented generation (RAG) to ensure its output is context-aware and grounded in real-time network data.

Threats

The project faces significant competition from robust, highly integrated, proprietary vendor platforms that dominate the enterprise network landscape. These vendors are actively beginning to integrate their own conversational AI and LLM features into their systems, which are Cisco, Juniper, Huawei, which could potentially erode CogniShield's competitive advantage in intelligent functionality over time. Finally, a major threat related to its core technology is the risk of LLM outputs suffering from hallucination or inaccuracy if the RAG context retrieval fails, potentially leading to configuration errors or unsafe network states if human review is circumvented.

3.3 Development Methodology — Waterfall SDLC



The CogniShield project adopts the **Waterfall Software Development Life Cycle (SDLC)** as the primary development methodology. The Waterfall model provides a structured, sequential, and documentation-driven approach that aligns well with the project's technical nature, safety requirements, and the constraints of an individual Final Year Project. Since this system deals with automated network configuration, topology interpretation, LLM-generated playbooks, validation pipelines, and remediation actions, strict stage-gated controls are essential. Unlike Agile, which prioritizes continuous iteration and dynamic requirement changes, the Waterfall model ensures that each stage is fully completed, reviewed, and validated before proceeding to the next. This reduces the risk of unsafe or unpredictable behaviours, especially when dealing with network infrastructure that may be sensitive to configuration errors (GeeksforGeeks, 2025).

The Waterfall approach is also suitable because the project's requirements can be clearly defined upfront through literature reviews, technical documentation, and research on network automation frameworks. The system's core components such as intent processing, topology visualization, validation logic, and execution workflows are well understood from the start, allowing for stable planning and systematic refinement. Moreover, as a solo developer in an academic environment, the Waterfall structure provides clear milestones, manageable workload segmentation, and predictable progress tracking, which is critical for academic supervision, evaluation, and reporting (Gazala, 2025).

3.3.1 Planning

The Planning phase forms the foundation of the entire CogniShield project by establishing a clear understanding of the project direction, technical expectations, and constraints. Since this system involves network automation, LLM-assisted playbook generation, telemetry analysis, and safety-critical workflows, careful preparation at the planning stage is essential to avoid unexpected issues in later phases (Malsam, 2025). As an individual student developer, this stage was used to build sufficient background knowledge and ensure that the subsequent phases could be carried out in a structured and confident manner.

The first activity in the planning stage involved carrying out a comprehensive review of academic literature, technical documentation, online research materials, and industry standards related to intent-based networking, RAG pipelines, and large language model automation in network operations (Xavier_Elliot, 2025). Commonly used telemetry technologies such as SNMP, gNMI, and model-driven streaming telemetry were also studied, as these form the basis of monitoring and automated remediation in modern networks (Flores, 2021). Through this review, the principles, best practices, and design considerations that would shape the architecture of CogniShield were identified.

In parallel, multiple vendor ecosystems were examined, including Cisco IOS, Juniper JunOS, Huawei VRP, et cetera, along with their associated management protocols such as NETCONF and RESTCONF (Juniper Networks, n.d.). This step helped to understand the different command structures, configuration styles, and operational verbs required for generating vendor-agnostic playbooks. By collecting sample device inventories, operating system versions, IP management interfaces, and authentication methods, realistic constraints for the system were defined and an initial dataset was prepared for functional testing later in the project.

Risk assessment formed another important part of the planning stage. Because the system interacts with network devices and produces configuration changes, possible threats such as prompt injection attacks, unsafe or ambiguous LLM outputs, misconfigured credentials, or unintentional device changes caused by flawed playbooks were analysed (OWASP Top 10 for LLM & Generative AI Security, 2025). To reduce these risks, mitigation strategies were documented that emphasise least-privilege access, the separation of management networks,

mandatory validation steps, approval workflows, and the use of static validators before any configuration is applied (Steven, 2025).

After developing a solid understanding of the domain and risks, a complete project schedule was constructed using a Gantt chart and a module-based Work Breakdown Structure. These tools helped organise tasks realistically across the available months and ensured that each SDLC stage received enough time for research, development, and testing (Landau, 2025). As this is a solo project, the planning stage also required the definition of learning milestones, particularly for areas such as NetBox integration, Ansible automation, prompt engineering, and topology processing, to ensure that all required tasks could be completed confidently.

Finally, the planning phase concluded with the definition of measurable success indicators. These include evaluating the accuracy of LLM-generated playbooks against manually written baselines, measuring the time required to detect and remediate incidents during pilot testing, and ensuring that automatically generated actions meet safety and reliability thresholds. This structured approach to planning ensures that the entire development process proceeds in a disciplined manner and that the final system is both functional and dependable.

3.3.2 Analysis

The Analysis phase converts the high-level plans into clear and testable requirements. It also produces the main artefacts that guide the design and implementation of CogniShield. For this single-developer Final Year Project, this stage was treated as the point where ideas gathered from literature were turned into a formal Software Requirements Specification and a prioritised feature set that fits the available time and lab resources. Therefore, the analysis emphasised clarity, measurability, and traceability so that every later design decision can be traced back to a verified requirement (TutorialsCampus, n.d.).

The first activity was to identify and document the functional requirements from the planning findings and previous studies. Based on operational scenarios, detailed use cases were written to reflect realistic operator workflows. For example, these included submitting a natural-language intent, reviewing and approving a generated playbook, running an Ansible dry-run in EVE-NG, visualising planned versus discovered topology, and initiating a rollback. In addition, the formal Functional Requirements table and the full Use Case List are presented in subsection 3.3.2.1 and summarised in a companion Use Case Diagram to help readers understand how actors and capabilities relate. These artefacts form the main functional requirements that the implementation must satisfy and provide the basis for acceptance-test creation (Letaw, 2024).

Non-functional requirements were also given attention because performance, reliability, security, and auditability are important for any automation platform working with network infrastructure. Therefore, measurable and achievable targets were set for the student lab context. For example, acceptable latency was defined for the intent-to-playbook cycle, maximum false-positive rates were set for topology-drift detection during pilot runs, expected controller availability was specified for demonstration scenarios, and the minimum audit information for each execution was recorded, such as playbook checksum, RAG evidence, approver identity, and timestamps. The full Non-Functional Requirements list and the associated acceptance metrics are provided in subsection 3.3.2.2, allowing designers and testers to check exact thresholds during development and evaluation (GeeksforGeeks, 2025c).

Next, a practical step was to list the technical interfaces that the system must support. Using vendor documentation and community resources, command families and northbound API calls were identified for the device families used in the EVE-NG lab. For instance, basic

IOS/IOS-XE commands were listed for Cisco-like devices, JunOS-like syntax was used for Juniper devices, and a generic CLI template was created for vendor-agnostic operations. In addition, the telemetry and discovery inputs were specified, including LLDP/CDP snapshots, basic SNMP counters, and short Prometheus/InfluxDB time-series windows. This step helped define precise input/output contracts for modules and the minimal NetBox schema needed for the RAG context bundler (InfluxData, 2021).

To make the requirements executable, modelling activities were also carried out. UML diagrams were created for practical implementation. These included a use case diagram showing actors and high-level capabilities, activity diagrams showing the full closed-loop process (intent → RAG → LLM → validator → dry-run → approve → apply), and sequence diagrams for key interactions, such as the sequence from intent submission to post-execution verification. In addition, a lightweight class/data model was drafted to represent core entities, including intent records, playbook records, execution logs, snapshots, and audit entries. These models were included in the SRS so that downstream design work could rely on stable data contracts (What Is Unified Modeling Language (UML)?, n.d.).

Since the project uses generative models, the analysis also included a plan for RAG data sources and test datasets. The documentation specifies which NetBox snapshots, telemetry slices, configuration templates, and historical playbooks will be used as the contextual corpus for grounding the LLM. For the MVP, the plan uses small, curated bundles attached to each intent instead of a full vector-indexed store. At the same time, the SRS records the corpus format, retention policy, and refresh cadence to make sure grounding can be repeated in the lab (Ramlochan, 2025). A small set of scenario-based test inputs was also defined, including example intents and expected canonical remediation steps, to check the correctness of the generated playbooks.

Security and threat modelling was formalised in this stage. The initial risk register was extended with concrete threat scenarios, attack surfaces, and mitigation measures. For example, prompt-injection risks were addressed by restricting allowed LLM outputs through template enforcement, recording RAG evidence for each generation, and requiring a static validator plus dry-run before any live apply. Credential compromise risks were mitigated using minimum-privilege device accounts for lab demonstrations and Ansible Vault for secrets in this FYP context (Chauhan & Chauhan, 2025). Each mitigation was linked to an acceptance criterion to allow test cases to verify its effectiveness.

Finally, a requirements-to-test-case traceability matrix was created. Every functional and non-functional requirement was linked to one or more acceptance tests, the responsible module, and the expected pass/fail criterion. This matrix served as the main checklist during the Integration and Testing phases and is included in the SRS deliverable (*Requirements Traceability Matrix: Definition, Benefits, and Examples | Perforce Software*, n.d.). Additional outputs from the analysis stage included a glossary of domain terms, the updated risk register, a prioritised backlog of features, and the signed-off SRS document that serves as the project baseline.

Lastly, practical constraints and assumptions were clearly recorded. These included EVE-NG hardware limits, the decision to use hosted LLM APIs or lightweight local models to avoid GPU dependencies, student-only resource availability, and the choice to keep the initial validator rule-set small. Recording these constraints early helped ensure that design and implementation stayed realistic and achievable within the timetable and available infrastructure.

3.3.2.1 Functional Requirement Analysis - Use Case Diagram

1. Network Topology Visualization Module

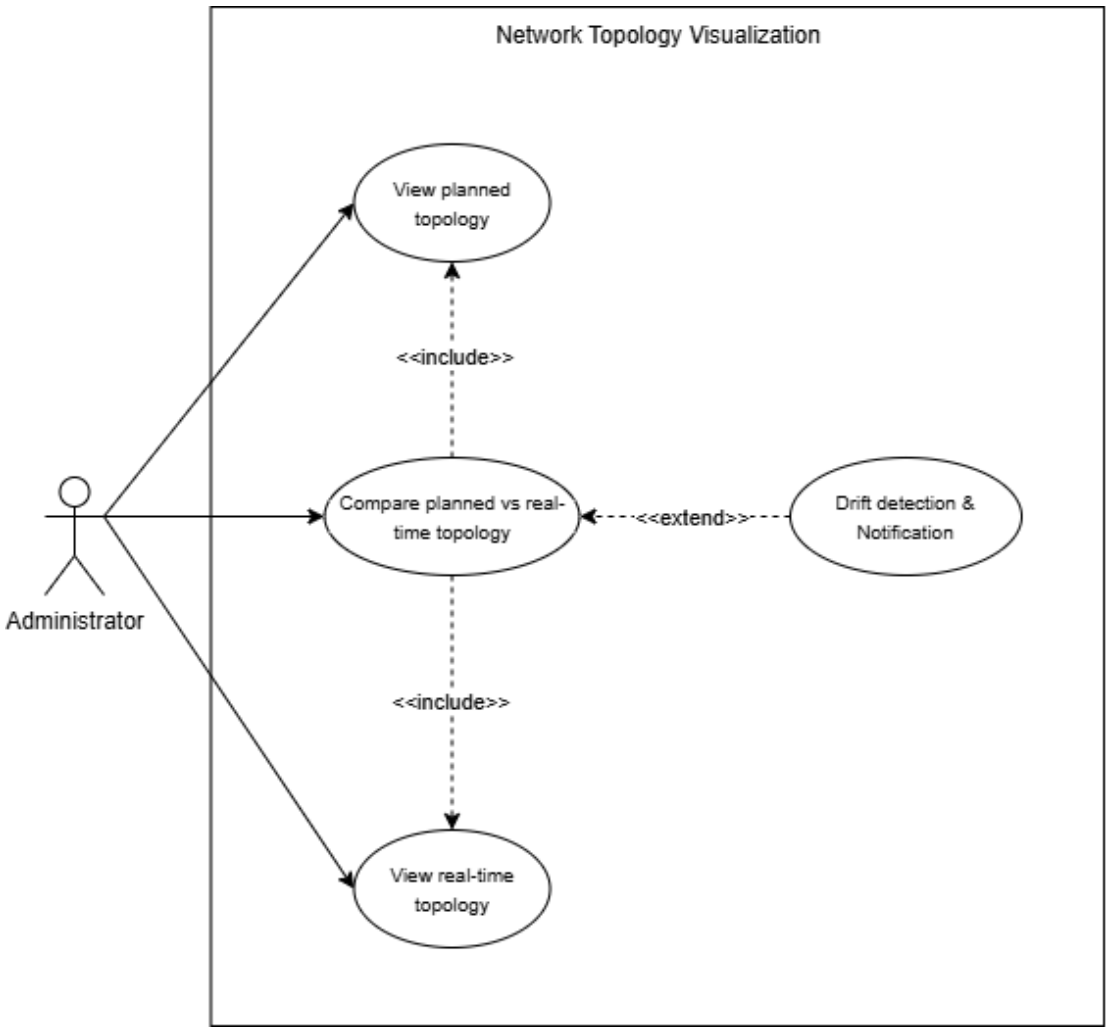


Figure 4: Network Topology Visualization Module Diagram

2. LLM Playbook Generation Module

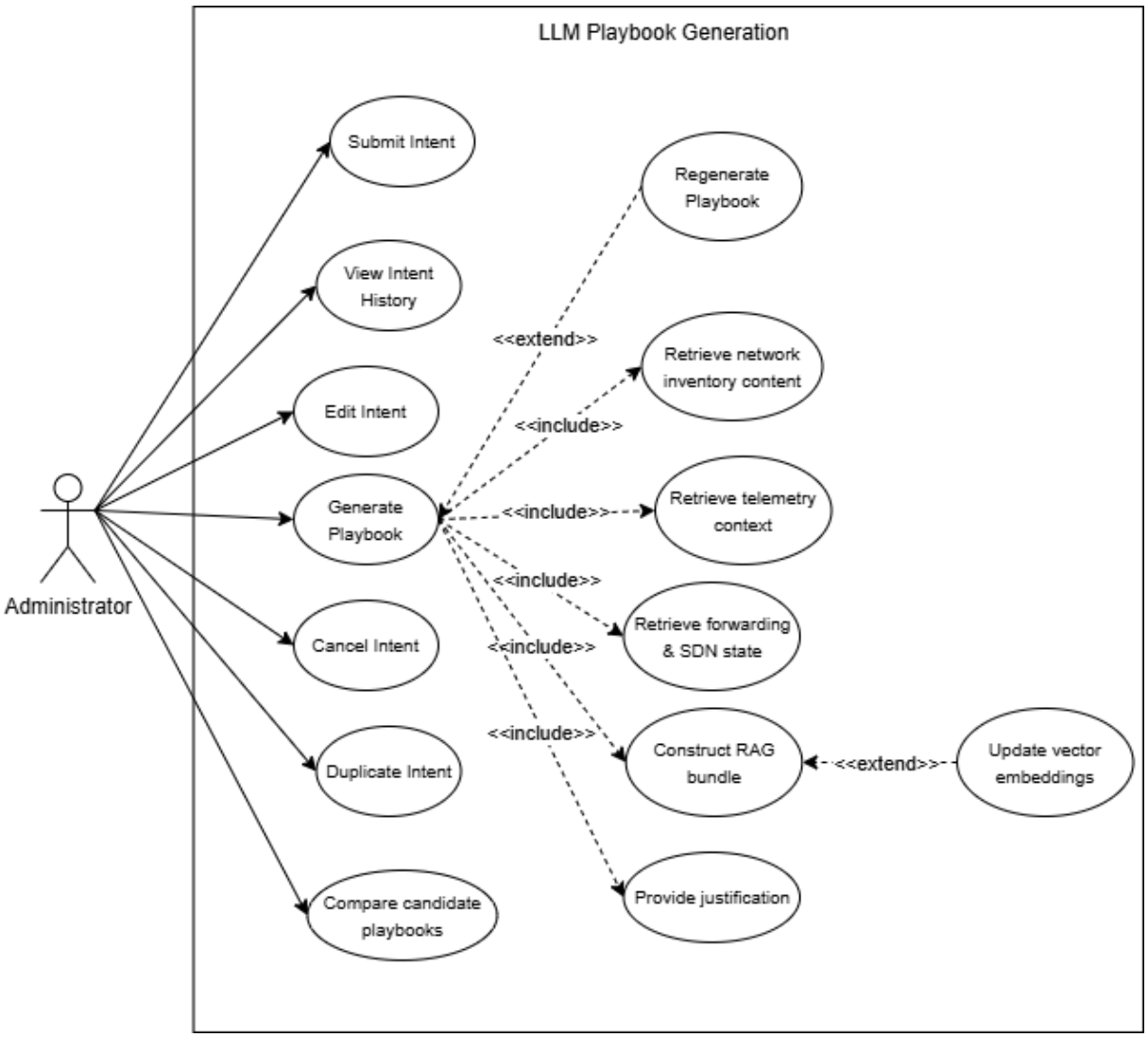


Figure 5: LLM Playbook Generation Module Diagram

3. Playbook Validation & Execution Module

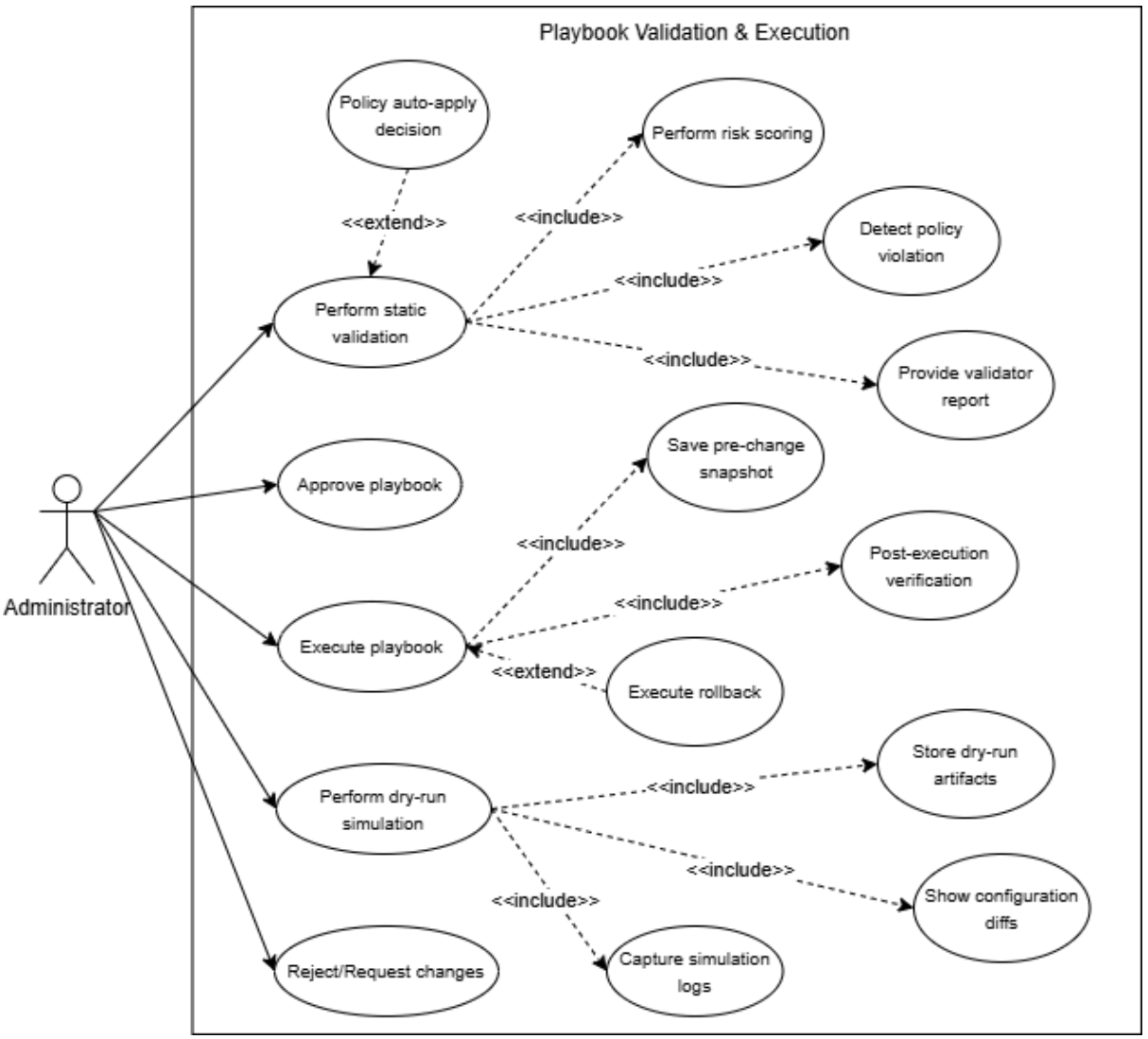


Figure 6: Playbook Validation & Execution Module Diagram

4. Alert & Automated Remediation Module

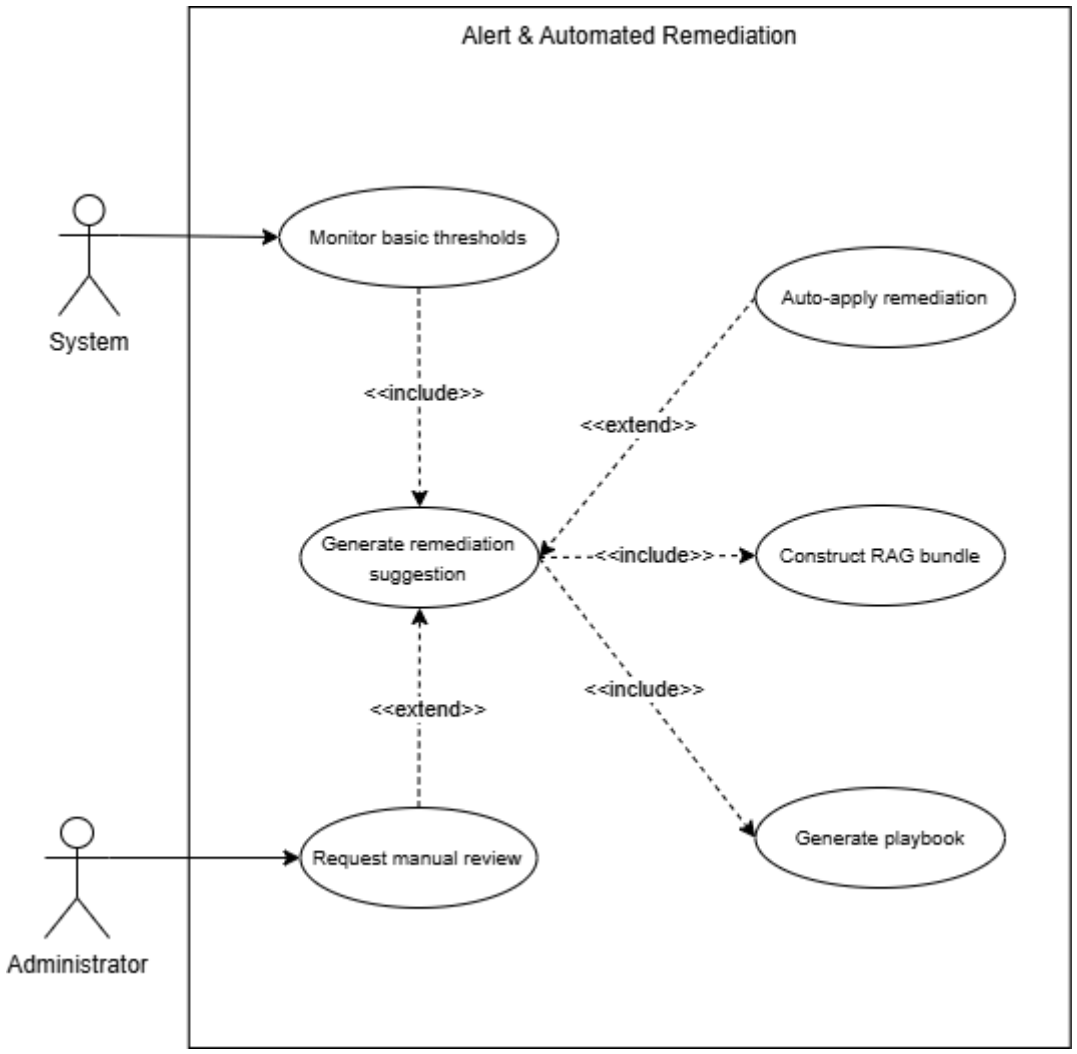


Figure 7: Alert & Automated Remediation Module Diagram

3.3.2.2 Functional Requirement - Use Case List

Module 1: LLM Playbook Generation

UC01 – Submit Intent

Use Case ID	UC01
Use Case Name	Submit Intent
Description	Administrator composes and submits a natural-language or structured instruction describing a desired network change. The system validates, stores and assigns a unique identifier to the intent, then enqueues it for further processing (RAG retrieval and LLM generation).
Actor	Administrator
Preconditions	User authenticated; intent composer UI available; NetBox and metadata services reachable for scope selection (optional).
Main Flow	1. Administrator opens intent composer. 2. Administrator types or pastes the intent text and optionally selects scope, priority and risk level. 3. System performs basic validation (non-empty, allowed characters, max length). 4. System saves the intent as an immutable record with metadata and a unique intent_id. 5. System enqueues the intent for RAG bundle construction and playbook generation. 6. UI acknowledges successful submission and shows status.
Alternative Flows	• If validation fails, UI shows error and request is not stored. • If connectivity to backend fails, the UI offers to save a local draft.
Postconditions	Intent stored in database with status "submitted" and queued for processing; user sees confirmation and request id.

UC02 – View Intent History

Use Case ID	UC02
Use Case Name	View Intent History
Description	Administrator browses a searchable, filterable list of previously submitted intents, including original text, generated playbooks, validation results, approvals and execution logs for audit and reuse.
Actor	Administrator
Preconditions	At least one intent record exists; user has read permission for intent history.
Main Flow	1. Administrator navigates to the intent history page. 2. System queries the metadata store and returns paged results. 3. Administrator filters/sorts (date, status, author, device scope). 4. Administrator selects an intent to view details (playbooks, diffs, approvals).
Alternative Flows	• If no intents found, system displays an empty state and guidance to submit new intent. • If database query fails, system shows an error and retry option.
Postconditions	Administrator can inspect historical intents and related evidence; selected records are available for cloning or comparison.

UC03 – Edit Intent

Use Case ID	UC03
Use Case Name	Edit Intent
Description	Administrator modifies an intent that is still a draft or pending generation (e.g., refine wording, change scope or risk level) to improve generation quality.

Actor	Administrator
Preconditions	Intent exists and its status allows editing (draft/pending); user has edit permission.
Main Flow	1. Administrator opens the draft intent. 2. Administrator updates text and/or structured parameters. 3. System validates changes. 4. System saves a new version and updates metadata (version, editor, timestamp). 5. If previously queued, system re-queues for generation with new content.
Alternative Flows	• If intent already progressed beyond editable state (e.g., generated playbook exists or execution in progress), system prevents edit and suggests clone/duplicate.
Postconditions	Updated intent version saved; processing pipeline uses the latest version for generation.

UC04 – Cancel Intent

Use Case ID	UC04
Use Case Name	Cancel Intent
Description	Administrator cancels an in-progress intent to stop generation, dry-run, or further processing.
Actor	Administrator
Preconditions	Intent is in a cancellable state (queued / generating / in dry-run); user has cancellation privilege.
Main Flow	1. Administrator selects cancel option. 2. System marks intent status as "cancelled" and stops queued jobs where possible.

	3. System records cancellation reason and actor. 4. System notifies administrator of successful cancellation.
Alternative Flows	• If job already completed, cancellation is not allowed and system indicates current status with next steps (rollback or follow-up).
Postconditions	Intent processing is halted, status set to cancelled, and audit entry recorded.

UC05 – Duplicate Intent

Use Case ID	UC05
Use Case Name	Duplicate Intent
Description	Administrator clones an existing intent to reuse or adapt it for another time or site, saving time and preserving provenance.
Actor	Administrator
Preconditions	Source intent exists and is visible to the user; user has create permission.
Main Flow	1. Administrator selects an existing intent. 2. Administrator chooses "Duplicate". 3. System creates a new draft intent prefilled with original text and metadata references. 4. Administrator edits draft and submits.
Alternative Flows	• If source intent is restricted, duplication is disallowed and system explains permission requirements.
Postconditions	New draft intent created with links to original; ready for editing or submission.

UC06 – Retrieve Network Inventory Context

Use Case ID	UC06
Use Case Name	Retrieve Network Inventory Context
Description	The system fetches device metadata and relationships (vendor, OS, model, interfaces, VLANs) from the canonical inventory (NetBox) to ground playbook generation.
Actor	System (RAG Engine)
Preconditions	NetBox accessible; intent includes scope or target devices; credentials available to query NetBox.
Main Flow	1. RAG engine receives intent scope. 2. System queries NetBox for relevant devices and relationships. 3. System selects top-k relevant inventory items and includes them in the context bundle. 4. Bundle stored or passed to LLM adaptor.
Alternative Flows	• If NetBox query times out, system retries or returns a partial bundle with warning. • If inventory stale, system attaches freshness metadata and flags potential risk.
Postconditions	An inventory context bundle is available and linked to the intent for grounding.

UC07 – Retrieve Telemetry Context

Use Case ID	UC07
Use Case Name	Retrieve Telemetry Context
Description	The system collects recent KPIs, alerts and time-series samples from Prometheus/InfluxDB to provide situational context that

	informs playbook generation.
Actor	System (RAG Engine)
Preconditions	Time-series DB reachable; telemetry collectors operational; intent scope defined.
Main Flow	1. RAG engine identifies telemetry metrics relevant to the target devices. 2. System queries Prometheus/InfluxDB for recent windows (e.g., last N samples). 3. System includes summarized KPI snapshots and spike indicators in the context bundle. 4. Bundle stored or passed to LLM adaptor.
Alternative Flows	• If telemetry missing or sparse, system notes absence and uses inventory-only grounding.
Postconditions	Telemetry context appended to the RAG bundle with timestamps and metric references.

UC08 – Retrieve Forwarding and SDN State

Use Case ID	UC08
Use Case Name	Retrieve Forwarding and SDN State
Description	The system queries SDN controller or forwarding tables to collect current forwarding state, VLAN databases and topology details needed for safe playbook generation.
Actor	System (RAG Engine)
Preconditions	SDN controller or forwarding API accessible; proper credentials; scope includes SDN-managed domains.
Main Flow	1. System queries SDN northbound API for forwarding/topology

	state. 2. System extracts relevant entries and includes them in the context bundle (e.g., routing table snippets, active VLANs). 3. System annotates bundle with controller timestamp and provenance.
Alternative Flows	• If SDN API unavailable, system logs absence and relies on LLDP/NetBox approximations.
Postconditions	Forwarding/state information included in the intent's context bundle.

UC09 – Construct RAG Bundle

Use Case ID	UC09
Use Case Name	Construct RAG Bundle
Description	The RAG engine compiles selected inventory, telemetry and forwarding items into a compact, structured context bundle attached to the intent and used to ground the LLM.
Actor	System (RAG Engine)
Preconditions	Inventory, telemetry and forwarding queries completed (UC06–UC08).
Main Flow	1. System ranks retrieved artifacts by relevance (top-k). 2. System composes a JSON context bundle with metadata, excerpts, and pointers to source records. 3. System stores the bundle and links it to the intent record; bundle is provided to the LLM adaptor.
Alternative Flows	• If bundle size exceeds limits, system applies summarization or truncation policies and annotates omitted items.
Postconditions	A compact RAG bundle is saved and associated with the intent for

	the LLM generation step.

UC10 – Update Vector Embeddings

Use Case ID	UC10
Use Case Name	Update Vector Embeddings
Description	When topology or inventory changes, the system refreshes embeddings or regenerates context bundles so future retrievals remain accurate.
Actor	System
Preconditions	Change events (inventory update, new snapshot) are recorded; vector store is available.
Main Flow	1. Background job detects inventory/topology change. 2. System regenerates affected document chunks and computes new embeddings. 3. Updated embeddings are stored into the vector DB and mapping indexes refreshed. 4. System logs update activity and timestamps.
Alternative Flows	• If vector DB is temporarily unavailable, updates are queued and retried.
Postconditions	Embeddings and retrieval indexes are refreshed and reflect current topology/inventory.

UC11 – Generate Playbook

Use Case ID	UC11
Use Case Name	Generate Playbook
Description	The LLM adaptor consumes the natural-language intent and the RAG bundle to synthesize one or more candidate Ansible playbooks or CLI sequences suitable for the target devices.
Actor	System (LLM Adapter)
Preconditions	Intent and RAG bundle available; LLM endpoint (local or hosted) reachable.
Main Flow	1. Adapter constructs a templated prompt including intent, RAG evidence and generation constraints. 2. Adapter sends prompt to LLM and requests playbook output (Ansible/CLI). 3. Adapter receives output, performs basic parsing and sanitization, and stores candidate(s) as Playbook Record(s) with provenance and checksum. 4. System surfaces candidates to the UI for review.
Alternative Flows	• If LLM times out or returns invalid format, adapter returns an error with LLM logs and suggests regeneration. • If LLM returns potentially dangerous commands, static validator may auto-flag or block before operator sees it.
Postconditions	One or more candidate playbooks are stored and linked to the intent for validation and review.

UC12 – Provide Justification

Use Case ID	UC12
Use Case Name	Provide Justification

Description	For each generated candidate, the system produces a concise human-readable justification that cites RAG evidence and explains why the chosen actions are appropriate.
Actor	System (LLM Adapter)
Preconditions	Candidate playbook exists; RAG bundle is available for provenance.
Main Flow	1. Adapter requests or extracts a justification from the LLM output. 2. System associates the justification text with the corresponding playbook record. 3. UI displays playbook plus justification for operator review.
Alternative Flows	• If LLM fails to produce a justification, a template justification is generated from the RAG evidence summary.
Postconditions	Playbook records include a human-readable justification and provenance pointers.

UC13 – Regenerate Playbook

Use Case ID	UC13
Use Case Name	Regenerate Playbook
Description	Administrator requests a new candidate playbook by modifying constraints, changing prompt templates, or editing the intent; the system re-runs the generation step and stores new candidates.
Actor	Administrator
Preconditions	Original playbook generation completed and user has edit/regen permission.
Main Flow	1. Administrator modifies generation parameters or intent text. 2. Administrator clicks "Regenerate".

	3. System reconstructs the prompt and calls the LLM adaptor. 4. New candidate(s) stored as additional playbook records with separate provenance. 5. UI allows side-by-side comparison.
Alternative Flows	• If regeneration repeatedly fails, system suggests alternative prompt templates or manual playbook authoring.
Postconditions	New playbook candidates available and previous candidates preserved for audit.

UC14 – Compare Candidate Playbooks

Use Case ID	UC14
Use Case Name	Compare Candidate Playbooks
Description	Administrator views multiple generated playbooks side-by-side, compares diffs and provenance, and selects the preferred candidate to pass to validation.
Actor	Administrator
Preconditions	At least two candidate playbooks exist for the intent; user has view/compare permission.
Main Flow	1. Administrator opens the playbook comparison interface. 2. System fetches selected playbook records and computes textual diffs and a side-by-side view. 3. System shows provenance and justification for each candidate. 4. Administrator selects the preferred candidate and forwards it to the validation pipeline.
Alternative Flows	• If playbooks are already identical, system indicates no meaningful differences and suggests regen or manual edit.
Postconditions	Preferred playbook selected and queued for validation; comparison metadata stored in the intent history.

Module 2: Network Topology Visualization

UC15 – View Planned Topology (Design / Draft)

Use Case ID	UC15
Use Case Name	View Planned Topology (Design / Draft)
Description	This use case allows the administrator or network planner to view the intended or planned network topology that has been designed and stored in the inventory system. The topology represents the expected network structure before deployment.
Actor	Administrator
Preconditions	Planned topology data exists in NetBox; user is authenticated and has access rights
Main Flow	1. User selects the planned topology view from the GUI. 2. System retrieves design-layer topology data from NetBox. 3. Nodes, links, VLANs and device roles are rendered visually. 4. User explores topology using zoom, filter or layer controls.
Alternative Flows	If no planned topology is available, the system displays a notification requesting the user to create or upload a design first.
Postconditions	Planned topology is successfully displayed and available for reference or comparison.

UC16 – View Real-Time Topology (Discovered)

Use Case ID	UC16
Use Case Name	View Real-Time Topology (Discovered)
Description	This use case enables the administrator to view the current, discovered network topology based on live data collected from

	inventory sources, discovery protocols and SDN controllers.
Actor	Administrator
Preconditions	Network devices are reachable; discovery data is available; telemetry services are running
Main Flow	1. User selects live topology view. 2. System queries NetBox, SDN APIs and discovery protocols such as LLDP/CDP. 3. Live topology is constructed and rendered in the GUI. 4. Interface status and basic link metrics are shown.
Alternative Flows	If live data is incomplete or unavailable, the system displays partial topology with warning indicators.
Postconditions	Current network topology is visualized for monitoring and analysis.

UC17 – Compare Planned vs Real-Time Topology (Topology Diff)

Use Case ID	UC17
Use Case Name	Compare Planned vs Real-Time Topology
Description	This use case allows the system to compare the intended network design with the discovered live topology to identify mismatches or configuration drift.
Actor	Administrator
Preconditions	Both planned topology and live topology data are available
Main Flow	1. User selects topology comparison mode. 2. System loads planned and live topology snapshots. 3. Differences such as missing links or VLAN mismatches are computed. 4. Visual diff overlays are displayed in the topology view.

Alternative Flows	If one topology snapshot is missing, comparison is disabled with an explanatory message.
Postconditions	Topology differences are clearly identified and can be used for troubleshooting or remediation planning.

UC18 – Drift Detection and Topology-Change Notification

Use Case ID	UC18
Use Case Name	Drift Detection and Topology-Change Notification
Description	This use case enables automatic detection of topology or configuration drift and notifies the operator when predefined thresholds are exceeded.
Actor	System (Primary), Administrator (Secondary)
Preconditions	Monitoring and discovery services are active and drift rules are configured
Main Flow	1. System periodically checks topology and configuration state. 2. Differences from the planned state are detected. 3. Drift threshold is evaluated. 4. Incident record and notification are generated.
Alternative Flows	Minor drift below threshold is logged without triggering notification.
Postconditions	Operator is informed of topology drift and can initiate further investigation or remediation.

Module 3: Playbook Validation & Execution

UC19 – Perform Static Validation

Use Case ID	UC19
Use Case Name	Perform Static Validation
Description	The system validates the generated playbook using syntax checks, schema validation, vendor-specific linting rules, and predefined governance policies before any execution is allowed.
Actor	System (Validator Engine)
Preconditions	Candidate playbook exists and has been selected for validation; validation rules are configured.
Main Flow	1. System receives selected playbook. 2. Validator parses the playbook structure and syntax. 3. Vendor-specific rules and policy checks are applied (e.g., VLAN overlap, reserved resources). 4. Validation results are collected and classified as errors or warnings.
Alternative Flows	• If parsing fails, validation stops and a fatal error is reported.
Postconditions	Validation results are stored and linked to the intent for review.

UC20 – Perform Risk Scoring

Use Case ID	UC20
Use Case Name	Perform Risk Scoring
Description	The system calculates a risk score for the playbook based on device criticality, number of affected nodes, scope of changes, and historical execution outcomes.

Actor	System
Preconditions	Static validation completed; playbook metadata available.
Main Flow	1. System evaluates the number and type of devices affected. 2. Device roles (core, distribution, access) are weighted. 3. Historical success/failure data is referenced. 4. A composite risk score is generated and stored.
Alternative Flows	• If historical data is unavailable, a conservative default weighting is applied.
Postconditions	Risk score attached to the playbook and used for approval decisions.

UC21 – Policy Decision for Auto-Apply

Use Case ID	UC21
Use Case Name	Policy Decision for Auto-Apply
Description	The system evaluates whether the playbook can be automatically applied or requires manual approval based on policy rules and risk score.
Actor	System (Policy Engine)
Preconditions	Risk score computed; governance policies configured.
Main Flow	1. System compares risk score against policy thresholds. 2. Device and operation type are checked against allow/deny lists. 3. Decision is made to allow auto-apply or require approval.
Alternative Flows	• If policy rules conflict, the stricter rule is applied.
Postconditions	Playbook marked as “auto-apply eligible” or “manual approval required”.

UC22 – Detect Policy Violation

Use Case ID	UC22
Use Case Name	Detect Policy Violation
Description	The system detects violations of governance policies such as restricted devices, forbidden commands, or reserved network resources.
Actor	System
Preconditions	Static validation and policy checks running.
Main Flow	1. System evaluates playbook against governance rules. 2. Violations are identified and categorized. 3. Execution is blocked or restricted based on severity.
Alternative Flows	• Minor violations may allow override with explicit justification.
Postconditions	Violations recorded and surfaced to the operator.

UC23 – Provide Validator Report

Use Case ID	UC23
Use Case Name	Provide Validator Report
Description	The system presents a structured validation report summarizing errors, warnings, risk score, and policy findings for operator review.
Actor	System
Preconditions	Validation and risk scoring completed.
Main Flow	1. System compiles validation, risk, and policy results. 2. Report is formatted into a readable UI view. 3. Operator reviews findings and recommendations.
Alternative Flows	• If report generation fails, raw logs are displayed.

Postconditions	Operator has sufficient information to approve or reject the playbook.

UC24 – Perform Dry-Run Simulation

Use Case ID	UC24
Use Case Name	Perform Dry-Run Simulation
Description	The system executes the playbook in check mode against a sandbox or EVE-NG lab to simulate changes without affecting live devices.
Actor	System
Preconditions	Playbook validated; dry-run permitted by policy; lab environment available.
Main Flow	1. System launches playbook using Ansible check mode. 2. Target lab devices are contacted. 3. Simulated changes are evaluated. 4. Outputs and diffs are collected.
Alternative Flows	• If lab unreachable, dry-run is skipped with warning.
Postconditions	Dry-run results stored for review.

UC25 – Show Configuration Diffs

Use Case ID	UC25
Use Case Name	Show Configuration Diffs
Description	The platform displays before-and-after configuration differences generated during dry-run simulation.
Actor	Administrator
Preconditions	Dry-run completed; diff artifacts available.

Main Flow	1. Administrator opens diff view. 2. System renders per-device configuration changes. 3. Summaries highlight key modifications.
Alternative Flows	• If no changes detected, system shows “no-op” result.
Postconditions	Administrator understands expected impact before approval.

UC26 – Capture Simulation Logs

Use Case ID	UC26
Use Case Name	Capture Simulation Logs
Description	The system captures task-level execution logs, warnings, and errors generated during dry-run simulation for auditing and troubleshooting.
Actor	System
Preconditions	Dry-run execution performed.
Main Flow	1. System collects logs from Ansible Runner. 2. Logs are timestamped and tagged per device/task. 3. Logs are stored with the intent record.
Alternative Flows	• Partial logs stored if execution is interrupted.
Postconditions	Simulation logs archived and accessible for review.

UC27 – Store Dry-Run Artifacts

Use Case ID	UC27
Use Case Name	Store Dry-Run Artifacts
Description	The system stores dry-run outputs such as diffs, logs, and summaries as part of the intent’s permanent audit trail.

Actor	System
Preconditions	Dry-run completed successfully or partially.
Main Flow	1. System packages diffs and logs. 2. Artifacts are saved in persistent storage. 3. Metadata links artifacts to intent and playbook.
Alternative Flows	• Storage failure triggers retry and warning.
Postconditions	Dry-run artifacts preserved for audit and rollback decisions.

UC28 – Approve Playbook

Use Case ID	UC28
Use Case Name	Approve Playbook
Description	A designated operator reviews validation and dry-run evidence and formally approves the playbook for live execution.
Actor	Administrator
Preconditions	Validation and dry-run completed; approval rights granted.
Main Flow	1. Operator reviews validator report and diffs. 2. Operator provides approval confirmation. 3. System records approval signature and timestamp.
Alternative Flows	• Approval denied if mandatory checks are incomplete.
Postconditions	Playbook marked as approved and queued for execution.

UC29 – Reject or Request Changes

Use Case ID	UC29
Use Case Name	Reject or Request Changes

Description	Operator rejects the playbook or requests regeneration with comments explaining required changes.
Actor	Administrator
Preconditions	Playbook under review.
Main Flow	1. Operator selects reject or request-changes. 2. Operator enters review notes. 3. System updates playbook status and notifies generator stage.
Alternative Flows	• If no comment provided, system requests justification.
Postconditions	Playbook returned to generation stage or closed.

UC30 – Execute Playbook (Apply)

Use Case ID	UC30
Use Case Name	Execute Playbook (Apply)
Description	The system applies the approved playbook to live devices using controlled concurrency and monitored execution.
Actor	System
Preconditions	Playbook approved; devices reachable; rollback data prepared.
Main Flow	1. System launches Ansible Runner. 2. Playbook applied to target devices. 3. Execution logs streamed to UI.
Alternative Flows	• If device unreachable, execution continues on remaining devices and reports partial failure.
Postconditions	Configuration changes applied and execution results stored.

UC31 – Post-Execution Basic Verification

Use Case ID	UC31
Use Case Name	Post-Execution Basic Verification
Description	The system performs basic checks to confirm that intended changes (e.g., VLAN presence, interface state) are successfully applied.
Actor	System
Preconditions	Playbook execution completed.
Main Flow	1. System queries devices for expected state. 2. Results compared against intended configuration. 3. Pass/Fail recorded per device.
Alternative Flows	• Verification skipped if devices are temporarily unreachable.
Postconditions	Verification status stored and visible to operator.

UC32 – Save Pre-Change Snapshot

Use Case ID	UC32
Use Case Name	Save Pre-Change Snapshot
Description	The system captures minimal configuration snapshots before applying changes to support rollback.
Actor	System
Preconditions	Execution about to start; snapshot mechanism enabled.
Main Flow	1. System retrieves current device configuration snippets. 2. Snapshots are stored securely. 3. Snapshot linked to playbook execution record.
Alternative Flows	• If snapshot fails, system warns operator before proceeding.

Postconditions	Pre-change state available for rollback.

UC33 – Execute Rollback

Use Case ID	UC33
Use Case Name	Execute Rollback
Description	The system restores previous configurations using stored snapshots when execution fails or operator requests rollback.
Actor	System
Preconditions	Pre-change snapshot exists; rollback authorized.
Main Flow	1. Rollback playbook generated or retrieved. 2. System applies rollback to affected devices. 3. Post-rollback verification performed.
Alternative Flows	• Partial rollback if some devices unreachable.
Postconditions	Network restored to prior known state and rollback logged.

Module 4: Alert & Automated Remediation

UC34 – Monitor Basic Thresholds

Use Case ID	UC34
Use Case Name	Monitor Basic Thresholds
Description	The system continuously monitors basic, predefined network thresholds such as interface down events, high CPU or memory usage, packet errors, and link instability to detect potential network issues.
Actor	System
Preconditions	Telemetry sources (Prometheus, InfluxDB, SNMP/gNMI exporters) are configured and active; threshold policies are defined.
Main Flow	1. Monitoring component collects telemetry data at regular intervals. 2. Metrics are evaluated against configured thresholds. 3. When a threshold is exceeded, an alert event is triggered. 4. An incident record is created with relevant context.
Alternative Flows	• If telemetry data is temporarily unavailable, the system logs the gap and retries collection.
Postconditions	An incident record containing alert details and timestamps is stored for further processing.

UC35 – Notify/Alert Operator

Use Case ID	UC35
Use Case Name	Notify/Alert Operator
Description	This use case describes how the system notifies the operator when an alert is raised or when a remediation action is proposed or

	executed, ensuring operator awareness of network conditions and system activities.
Actor	System
Preconditions	An alert or remediation event has been generated, and the operator has access to the CogniShield dashboard.
Main Flow	1. The system detects an alert or remediation event. 2. An event record is created with severity and affected resources. 3. The system generates a notification message. 4. The notification is delivered to the operator interface. 5. The operator reviews the notification and accesses related details.
Alternative Flows	The operator is informed of the event, and the notification is logged for auditing.
Postconditions	• Multiple alerts are grouped into a single notification. • Low-severity events are logged without active notification. • Notification delivery failure is recorded and retried.

UC36 – Generate Remediation Suggestion

Use Case ID	UC36
Use Case Name	Generate Remediation Suggestion
Description	The system generates a suggested remediation playbook using the LLM based on the detected alert and its associated network context. This suggestion is intended for operator review in the MVP stage.
Actor	System
Preconditions	An incident record exists; inventory, topology, and telemetry context are available.
Main Flow	1. System assembles a compact incident context from telemetry,

	NetBox, and topology data. 2. The RAG pipeline grounds the context. 3. LLM generates a candidate remediation playbook. 4. A human-readable justification is produced for the proposed actions.
Alternative Flows	• If insufficient context is available, the system generates a conservative suggestion or requests manual intervention.
Postconditions	A remediation suggestion is attached to the incident and queued for review.

UC37 – Auto-Apply Remediation

Use Case ID	UC37
Use Case Name	Auto-Apply Remediation
Description	The system automatically applies a remediation playbook without manual approval when policy rules allow and the assessed risk is low.
Actor	System
Preconditions	Remediation suggestion exists; policy allows auto-apply; risk score is below configured threshold.
Main Flow	1. System evaluates remediation against auto-apply policies. 2. Static validation and optional dry-run are performed. 3. Playbook is executed on target devices. 4. Execution and verification results are recorded.
Alternative Flows	• If validation fails, auto-apply is aborted and manual review is requested.
Postconditions	Remediation is applied successfully or safely blocked, with full audit logs recorded.

UC38 – Request Manual Review

Use Case ID	UC38
Use Case Name	Request Manual Review
Description	The system requests operator review for remediation actions when the detected issue is high-risk, ambiguous, or restricted by policy.
Actor	Operator
Preconditions	Remediation suggestion generated; risk score exceeds threshold or policy disallows auto-apply.
Main Flow	1. System flags the incident as requiring manual review. 2. Operator is notified through the GUI or alert channel. 3. Operator reviews suggested remediation and supporting evidence. 4. Operator approves, modifies, or rejects the remediation.
Alternative Flows	• If operator does not respond within a defined time window, escalation rules may apply.
Postconditions	Remediation is either approved for execution, rejected, or deferred, with the decision logged.

3.3.2.3 Non-Functional Requirements

Non-functional requirements define the qualities, constraints, and performance expectations of CogniShield beyond its functional capabilities. These requirements are critical because the system interacts with live and simulated network environments, handles sensitive configuration data, and integrates AI-generated playbooks, where reliability, security, and traceability are paramount (GeeksforGeeks, 2025a). The following non-functional requirements are elaborated to ensure clarity, verifiability, and alignment with the project's scope and student lab constraints.

1. **Performance Requirements**

- **Intent-to-Playbook Latency:** The system must generate and return an LLM-assisted playbook within 10 seconds for a single intent in a pilot lab environment, assuming a local or hosted lightweight LLM model. This ensures operator responsiveness and real-time feasibility for campus-scale networks.
- **Topology Visualization Refresh Rate:** The live network topology view should refresh every 5 seconds using telemetry or discovery inputs, enabling near real-time monitoring without excessive load on the EVE-NG or testbed environment.
- **RAG Context Retrieval:** The retrieval of NetBox inventory, SDN controller state, and telemetry samples must complete within 2 seconds to maintain smooth LLM grounding and prevent operator idle waiting.

2. **Security Requirements**

- **Credential Management:** Device credentials and API keys must be stored securely using Ansible Vault or equivalent encryption, ensuring that only authorized components can access them during playbook execution.
- **Access Control:** Operator roles and approvals must be enforced according to defined workflows, preventing unauthorized execution of network changes.
- **LLM Output Control:** The system must implement template enforcement and static validation to prevent unsafe commands from reaching network devices, mitigating risks from LLM hallucinations.

- **Audit Trail Security:** All audit logs, including before/after snapshots and approval records, must be immutable and timestamped to support accountability and forensics in case of misconfiguration or errors.

3. **Maintainability**

- **Modular Architecture:** Each module such as Topology Visualization, LLM Playbook Generation, Validation & Execution, Alert & Remediation should be developed and maintained independently to simplify updates and debugging.
- **Documentation:** Clear, detailed inline code documentation and high-level module descriptions are required to enable future students or researchers to extend or adapt the system.
- **Version Control:** All source code, playbook templates, and configuration artifacts must be managed in a Git repository to track changes and facilitate rollback

4. **Usability and Operator Experience**

- **GUI Simplicity:** The interface must be concise, with evidence-centric dashboards, guided intent input panels, syntax-highlighted playbook displays, and easy-to-read telemetry overlays.
- **Error Feedback:** Validation errors, simulation diffs, or execution warnings should be presented clearly with actionable guidance to reduce operator confusion.
- **Training Time:** A new operator should be able to complete basic tasks, such as submitting intents and reviewing playbooks, within 30 minutes of instruction, making the system suitable for a university lab demonstration.

3.3.3 Design

The design of CogniShield mainly focuses on building a reliable and well-structured workflow that supports intent submission, LLM-based playbook generation, validation, approval, execution and automated remediation. Since the goal of the project is to automate network configurations safely, the design emphasises clear process flow, strict validation steps and proper auditability at every stage. The main architectural interactions are explained using three sequence diagrams, which shows how the system behaves under different operation scenarios.

Before going into the detailed flows for intent submission, validation, execution and remediation, it is helpful to first show the overall process of how CogniShield works as an integrated system. By understanding the overall workflow, it becomes easier to see how each component such as intent inputs, context retrieval, LLM grounding, playbook generation, validation mechanisms, approval steps and automated remediation connect together. Figure 8 below illustrates the overall workflow of the platform and gives a clearer picture of how these parts work together.

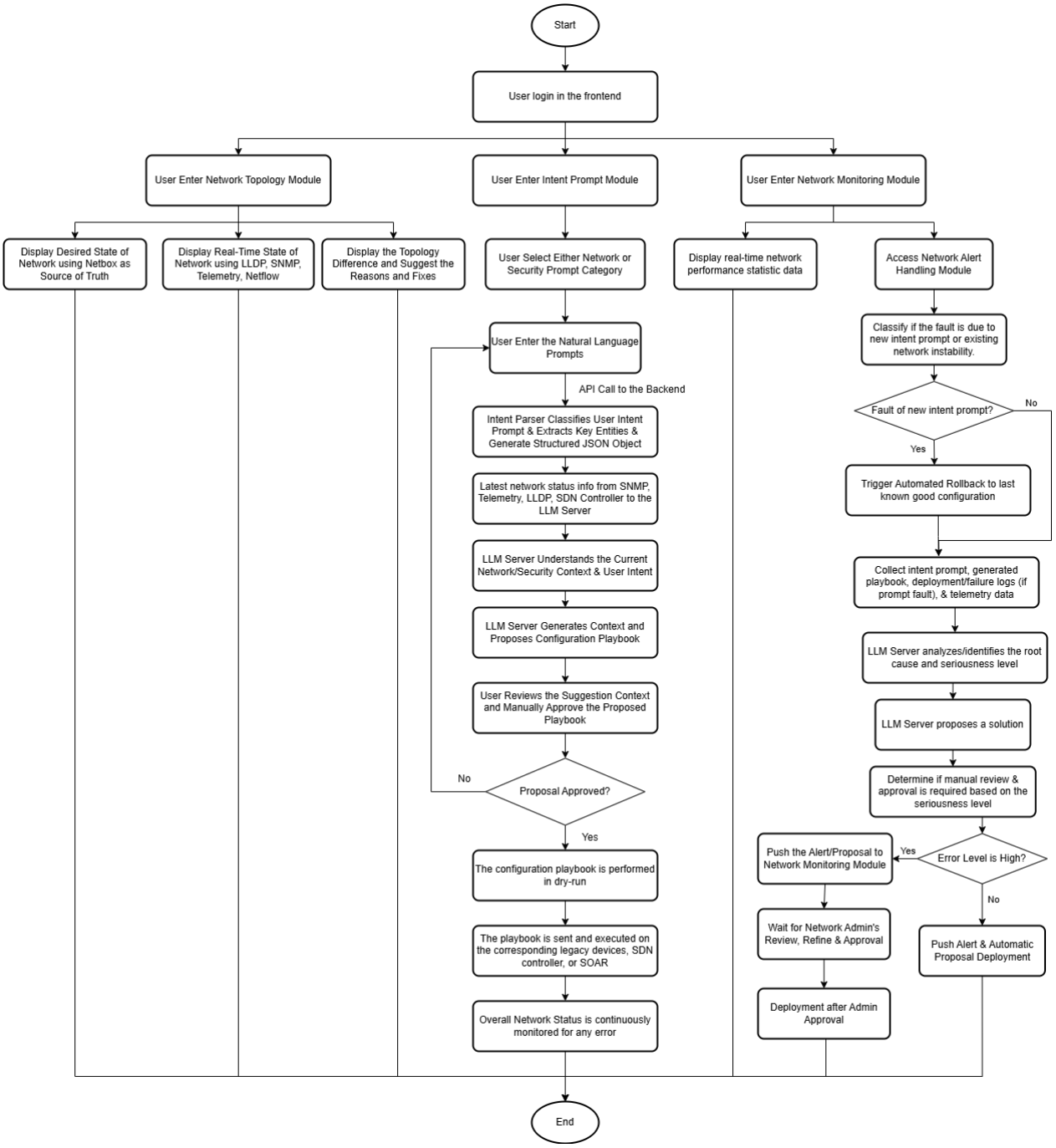


Figure 8: Overall Workflow of CogniShield

The system flow begins when an administrator submits an intent through the graphical user interface. Once received, the system logs the request and assigns a unique identifier. The retrieval pipeline then constructs a context bundle composed of NetBox inventory objects, SDN controller topology, forwarding or routing state and selected telemetry from the monitoring store. This bundle provides the grounding information required for the LLM to generate accurate and contextually appropriate configuration steps.

Using this context, the LLM adaptor produces a candidate playbook along with a human-readable justification explaining why each action is appropriate for the detected environment. Static validators analyse the playbook to detect syntactic issues, vendor-specific incompatibilities and logical conflicts such as overlapping VLAN IDs or IP address collisions. If enabled, the system executes a dry-run using Ansible check mode or through a sandbox such as EVE-NG. This generates diff outputs, warnings and execution traces that help the operator determine whether the generated actions are safe.

After validation, the operator enters the approval process. The interface shows the generated playbook, the evidence used during grounding and the results of any dry-run. The operator may choose to edit the playbook, regenerate it with modified instructions or approve it immediately. Once approval is given, the orchestrator applies the configuration to the target devices, records execution logs and performs post-execution verification. If the verification detects inconsistencies, the system prepares or triggers a rollback playbook. All these activities are logged to maintain a complete audit trail.

In addition to intent-driven automation, CogniShield also supports automated remediation based on alerts or anomalies detected from the telemetry pipeline. When the monitoring system raises an alert, a compact incident context is generated, which then flows through a reduced retrieval and grounding process. The LLM produces a remediation playbook, which is validated using the same safety constraints as standard intents. Depending on the operator configuration, the system may auto-apply the remediation, or require manual approval. Each remediation event is logged with full evidence and audit records.

Beyond the system flow, the GUI is designed to support clarity and reduce operator cognitive load. The landing dashboard highlights recent intents, top alerts and general campus health information. The intent composer provides a clean text-based input field with optional structured side panels for specifying target devices or risk level. After generation, the

playbook review interface displays syntax-highlighted YAML or CLI commands, affected devices derived from NetBox, telemetry snapshots and short LLM-generated justifications. High-risk changes may require multi-stage approvals. The audit section provides searchable access to past executions, configuration diffs and approval trails. The administration panel allows configuration of connectors, LLM endpoints, retrieval policies and operational settings.

The data design maintains a strong coherence between fast telemetry access and stable relational records. NetBox is the primary source of inventory information and is referenced directly instead of being fully duplicated. CogniShield stores persistent operational data in PostgreSQL, which mainly stores intents, generated playbooks, execution records, approvals and audit entries. On the other hand, time-series telemetry is stored in InfluxDB or Prometheus for efficient querying, while retrieval artefacts and embeddings are stored in a vector database such as Qdrant or ChromaDB. This arrangement ensures that grounding can be reconstructed accurately and that all system decisions remain auditable.

The relational structure is intentionally kept simple. A single intent may produce multiple candidate playbooks. A playbook may have multiple execution attempts. Each execution may generate several audit logs. Linking these entities ensures integrity while supporting scalable queries for evidence, history and traceability. In overall, the design ensures that CogniShield behaves predictably, can be verified at every major stage and preserves a complete audit chain from intent submission to remediation. By combining structured workflows, strict validation, clear GUI interaction and robust data modelling, the system provides a safe environment for testing network automation in a laboratory setting.

`

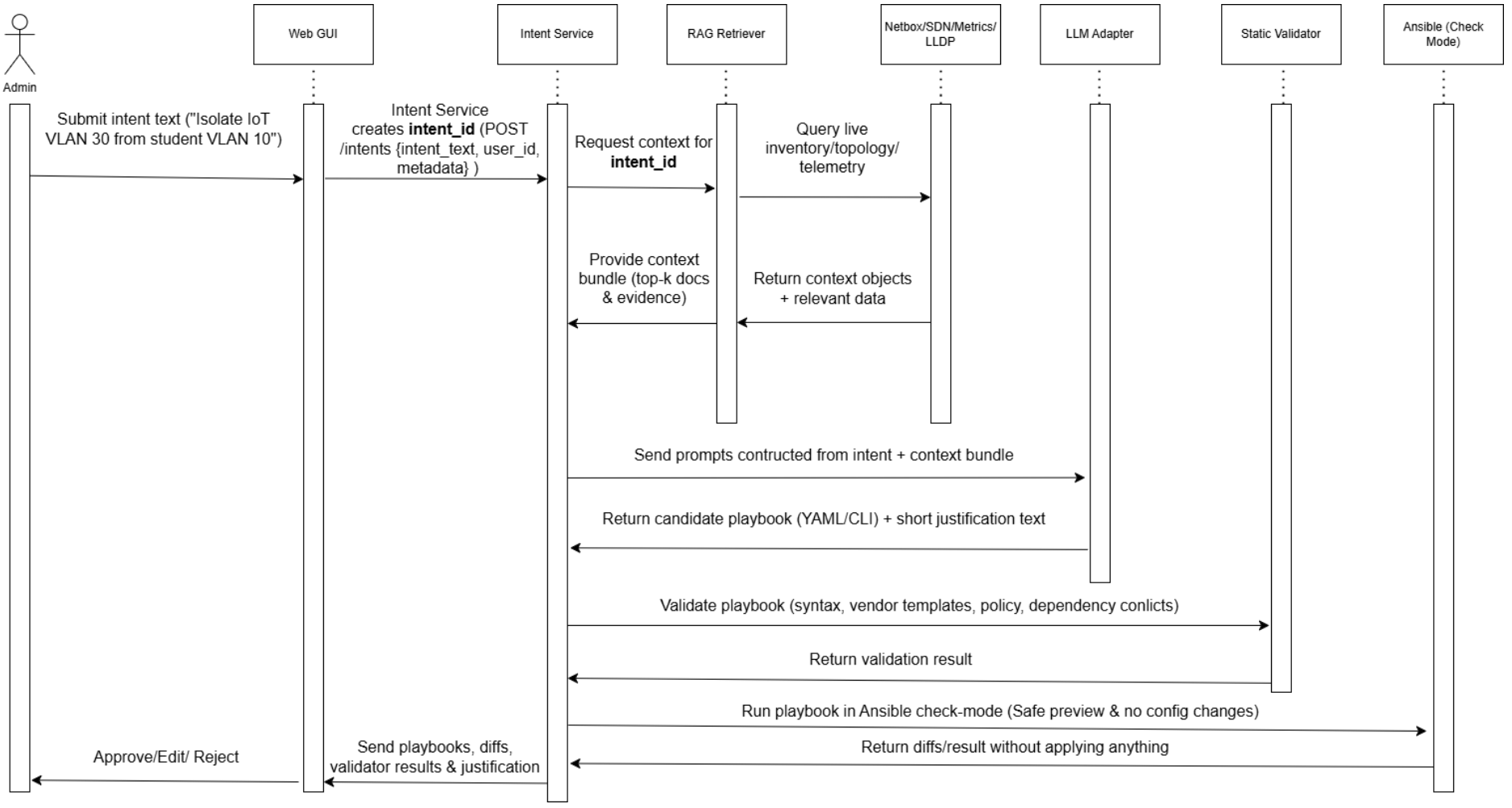


Figure 9: Intent Submission Sequence Diagram

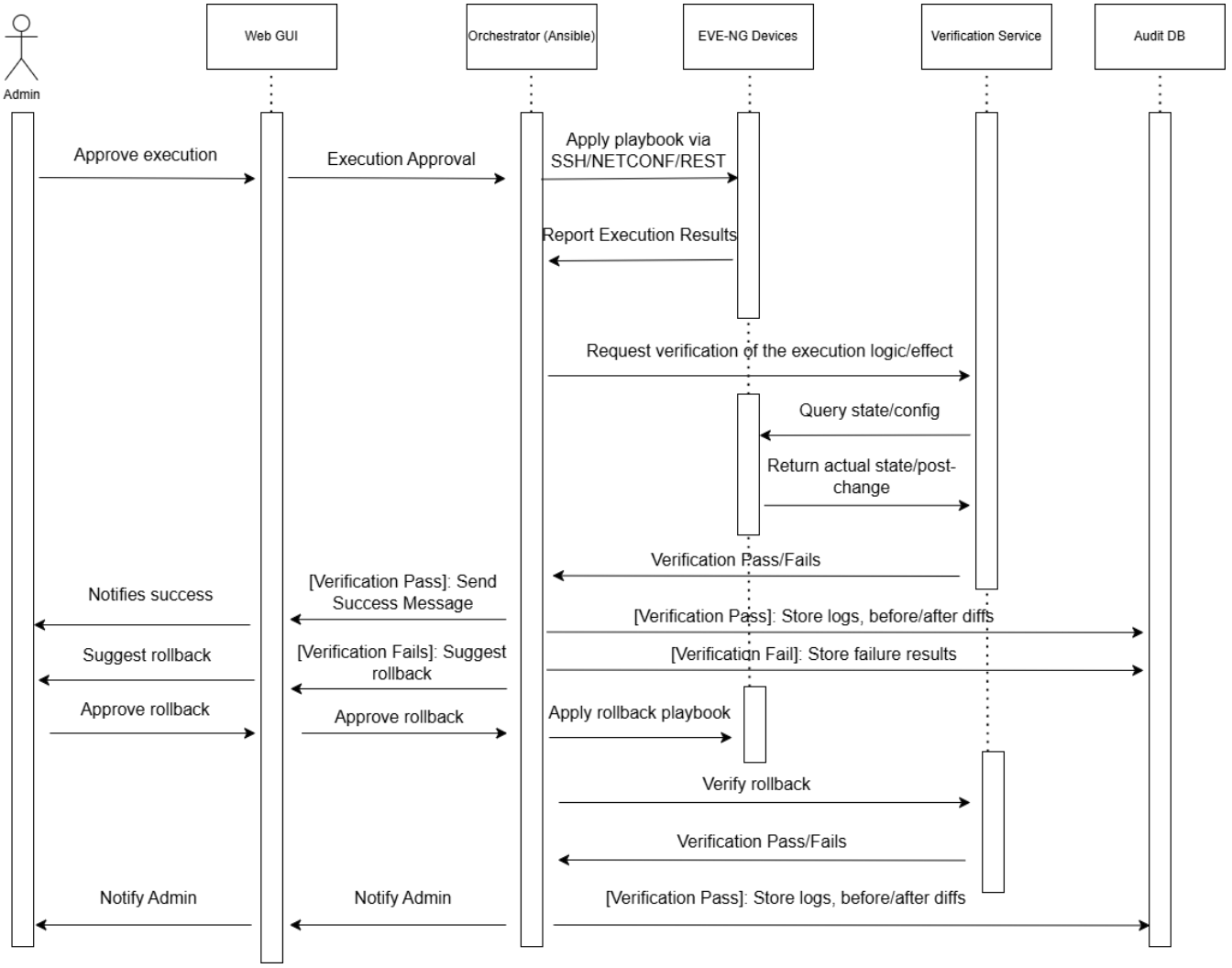


Figure 10: Approval, Validation and Execution Sequence Diagram

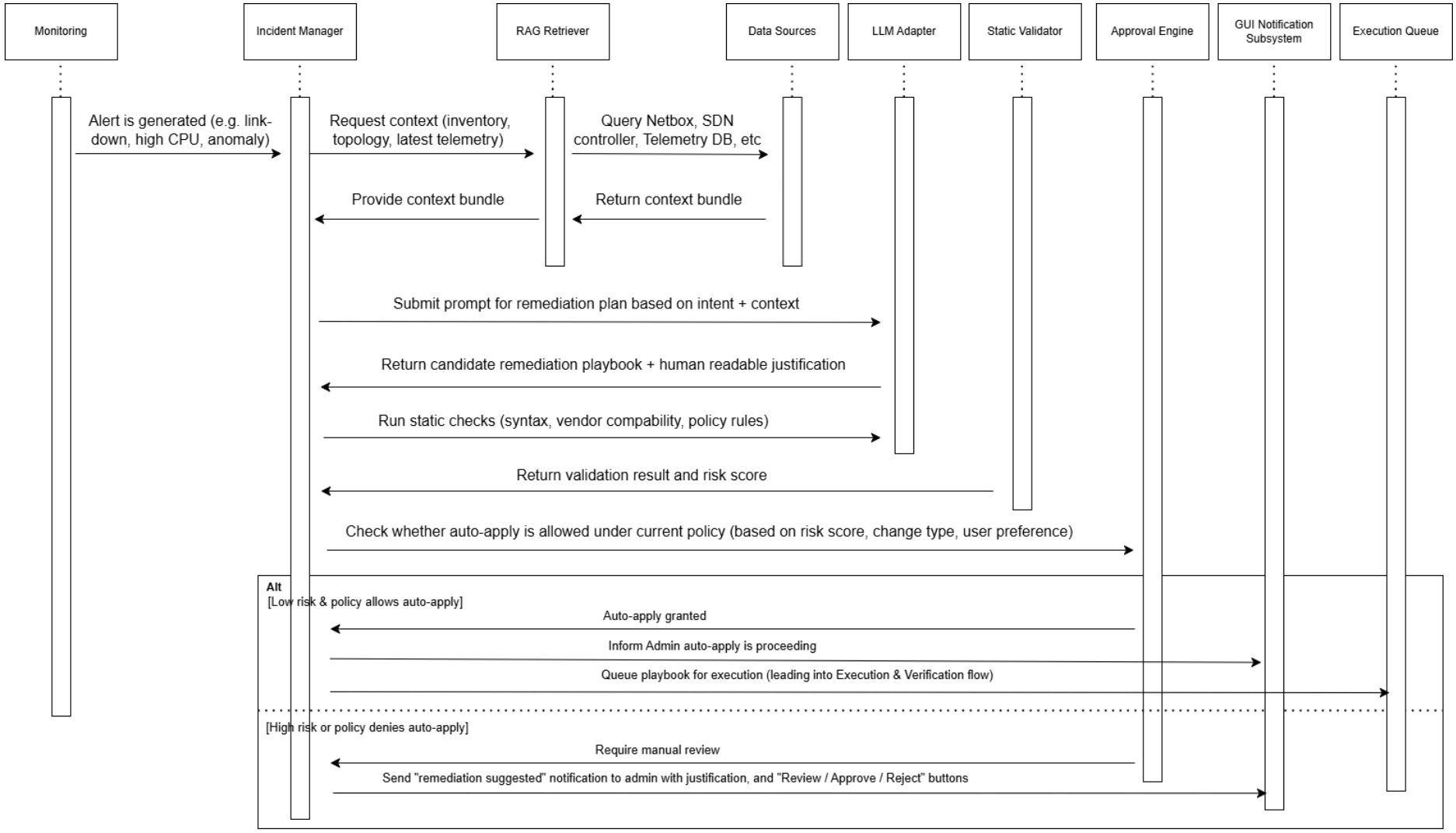


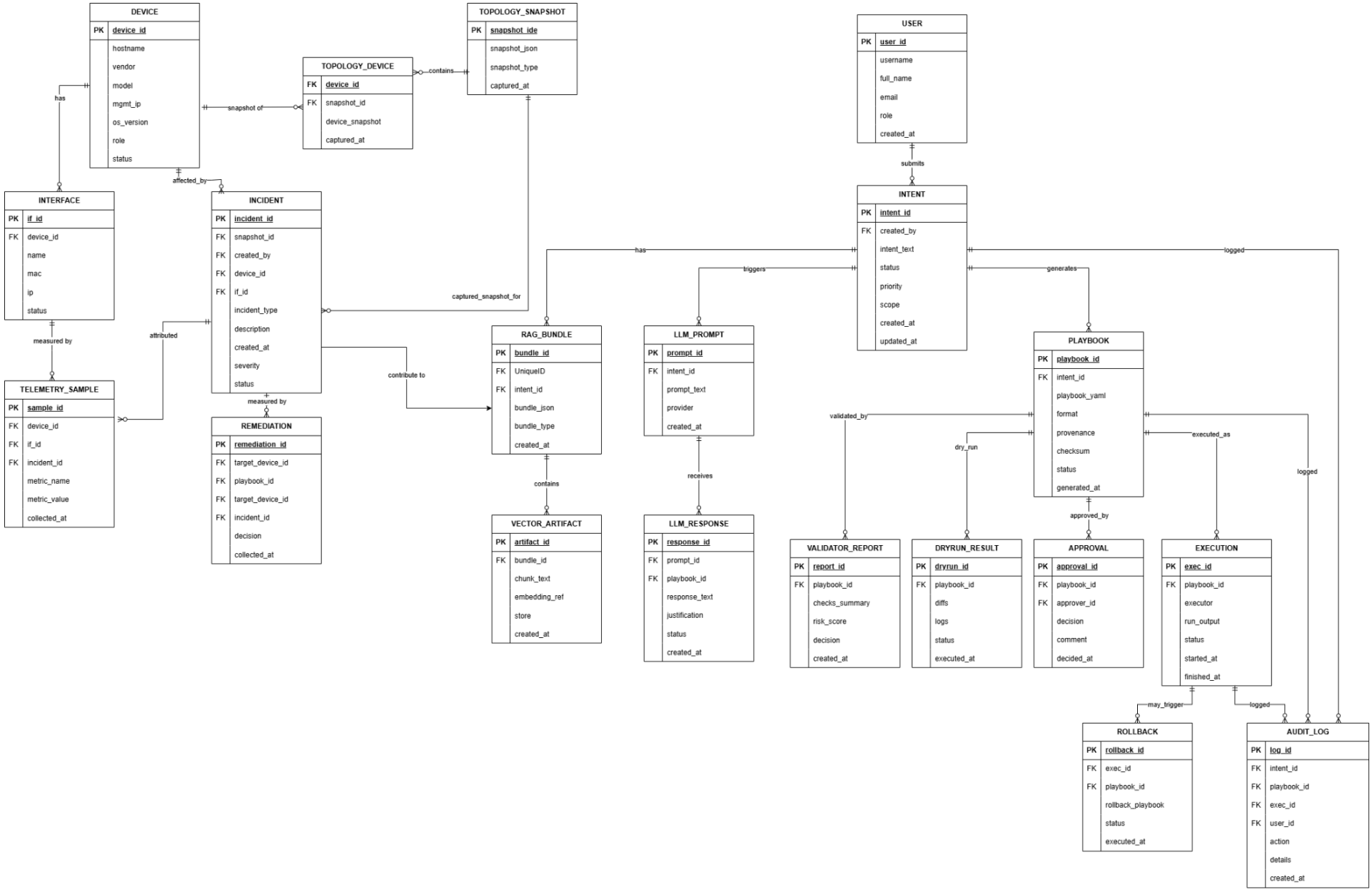
Figure 11: Alert and Automated Remediation Sequence Diagram

GUI Behaviour

The GUI is designed to be simple and focused on showing only the necessary information to make it easier for the operator to use. The landing dashboard shows a summary of campus health, including top alerts and recent intent statuses. The intent composer provides a simple text area for natural-language instructions, along with an optional side panel for structured fields like target site, device groups, priority, and risk level. After a playbook is generated, the review pane displays the commands in a readable format, lists the affected devices, and shows small telemetry charts for key metrics. The approval screen lets the operator edit, regenerate, or approve the playbook, while also showing any validation errors or dry-run results. Administration screens allow configuration of connectors, credentials, LLM endpoints, and retrieval settings. For each main screen, wireframes and behavior details are provided in the Design appendix and linked to the relevant use cases and acceptance criteria.

Database Schema and Data Stores

CogniShield keeps different types of data in separate storage systems to stay organized. NetBox is used as the main source for inventory information. Metadata and governance artifacts are kept in PostgreSQL with principal tables such as **intent** *(id, creator, raw_text, parsed_json, status, created_at)*, **playbook** *(id, intent_id, yaml_blob, checksum, provenance, risk_score, status)*, **execution** *(id, playbook_id, target_list, start_ts, end_ts, result_summary)*, *approval (id, playbook_id, approver_id, role, decision, timestamp)* and **audit_log** *(id, entity_ref, before_blob, after_blob, signature, timestamp)*. Aside from that, Telemetry and time-series data are stored in InfluxDB or Prometheus for efficient queries. On the other hand, RAG artifacts, which comprises document chunks, retrieval pointers and optional embeddings, are stored in a vector store (Qdrant or ChromaDB) with links back to Netbox and snapshot records.


DEVICE
PK device_id
hostname
vendor
model
mgmt_ip
os_version
role
status
TOPOLOGY_DEVICE
FK device_id
FK snapshot_id
device_snapshot
captured_at
TOPOLOGY_SNAPSHOT
PK snapshot_ide
snapshot_json
snapshot_type
captured_at
USER
PK user_id
username
full_name
email
role
created_at
INTERFACE
PK if_id
FK device_id
name
mac
ip
status
INCIDENT
PK incident_id
FK snapshot_id
FK created_by
FK device_id
FK if_id
incident_type
description
created_at
severity
status
INTENT
PK intent_id
FK created_by
intent_text
status
priority
scope
created_at
updated_at
RAG_BUNDLE
PK bundle_id
FK UniqueID
FK intent_id
bundle_json
bundle_type
created_at
LLM_PROMPT
PK prompt_id
FK intent_id
prompt_text
provider
created_at
PLAYBOOK
PK playbook_id
FK intent_id
playbook_yaml
format
provenance
checksum
status
generated_at
TELEMETRY_SAMPLE
PK sample_id
FK device_id
FK if_id
FK incident_id
metric_name
metric_value
collected_at
REMEDIATION
PK remediation_id
FK target_device_id
FK playbook_id
FK target_device_id
FK incident_id
decision
collected_at
VECTOR_ARTIFACT
PK artifact_id
FK bundle_id
chunk_text
embedding_ref
store
created_at
LLM_RESPONSE
PK response_id
FK prompt_id
FK playbook_id
response_text
justification
status
created_at
VALIDATOR_REPORT
PK report_id
FK playbook_id
checks_summary
risk_score
decision
created_at
DRYRUN_RESULT
PK dryrun_id
FK playbook_id
diffs
logs
status
executed_at
APPROVAL
PK approval_id
FK playbook_id
FK approver_id
decision
comment
decided_at
EXECUTION
PK exec_id
FK playbook_id
executor
run_output
status
started_at
finished_at
ROLLBACK
PK rollback_id
FK exec_id
FK playbook_id
rollback_playbook
status
executed_at
AUDIT_LOG
PK log_id
FK intent_id
FK playbook_id
FK exec_id
FK user_id
action
details
created_at
has
snapshot of
contains
affected_by
submits
measured by
attributed
measured by
contribute to
captured_snapshot_for
has
triggers
generates
logged
validated_by
dry_run
executed_as
logged
approved_by
contains
receives
may_trigger
logged

3.3.4 Implementation

The implementation phase translates the design into a working, reproducible system that can be demonstrated in a student laboratory and extended for future research. For this project, a pragmatic, Python-first stack is chosen because Python provides mature libraries for networking, automation and ML integration while remaining accessible for rapid development (*The Python Standard Library*, n.d.). The backend is implemented using modern Python, targeting a current stable release such as 3.14.x. It is designed as an HTTP and WebSocket service, with FastAPI preferred for its support of asynchronous operations and convenient handling of WebSocket connections (*FastAPI*, n.d.), while the frontend is a lightweight single-page application written in React so the user interface remains responsive while streaming telemetry and execution logs to the operator (*Quick Start – React*, n.d.). All services are containerised to make deployments repeatable, where Docker Compose are used for lab demonstrations and an optional Kubernetes configuration could be used for larger or future pilot deployments.

To bridge the high-level architecture shown in *Figure 12: Integration of CogniShield to Campus Network in High-Level Architecture View* with a concrete deployment suitable for laboratory demonstration, a low-level architecture mapping is provided next. This mapping will show where each logical service will run at runtime, whether as a container, a process, or inside a dedicated virtual machine and also explains how the Proxmox virtualization environment is used to simulate an actual multi-host deployment. In the student lab the Proxmox host will run multiple virtual machines, each VM responsible for a clear group of services, for example the backend API and orchestration, RAG/LLM and vector store, telemetry and monitoring stack, NetBox inventory, executor/validator, and the EVE-NG topology simulator. Inside each VM, finer-grained components could be containerised using Docker Compose if the time allows so that containers can be started, stopped and snapshotted together with the VM. This attempt of using two-level virtualization technologies which are virtual machines per major service group and also containers per component is meant to reproduce and simulate a realistic multi-host deployment while remaining operable and manageable on a single lab server in this final year project.

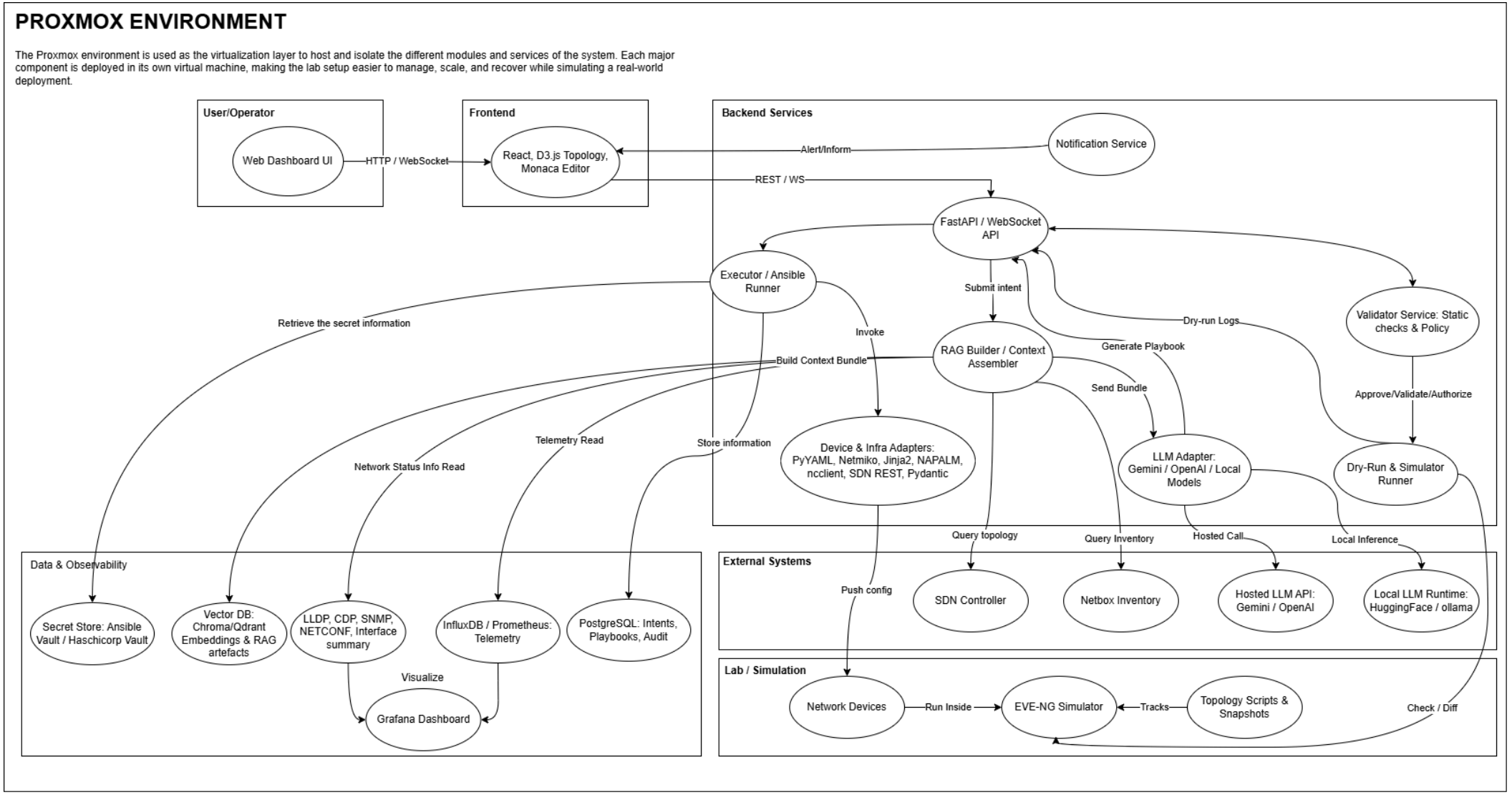


Figure 12: Low-Level CogniShield Architecture Diagram

The heart of the runtime is the RAG and LLM pipeline and the automation orchestrator. The RAG layer gathers short, authoritative context bundles from NetBox using pynetbox, SDN northbound endpoints, SNMP, and the telemetry store in Prometheus or InfluxDB (Netbox-Community, n.d., *Overview | Prometheus*, n.d., *InfluxDB OSS V2 Documentation*, n.d.). These bundles are kept compact on purpose, which include the most important inventory items, recent interface samples, and the directly connected neighbours. This helps the LLM understand the current state of the network. The system can work in two ways for using the LLM. It can either call hosted model APIs, such as Google's Gemini Open API or OpenAI, or it can also run local models using tools like Hugging Face runtimes or quantized runtimes, for example llama.cpp/ggml when on-premise inference is required (*Transformers*, n.d., Ggml-Org, n.d.). Every time the system generates a playbook, the system saves key details, including the prompt given to the LLM, the RAG evidence used, and the response from the model. This makes it easy to explain and track how the playbook was created.

In this project, playbooks are produced as Ansible YAML files or as device CLI commands. The automation engine runs these playbooks using ansible-runner, so they can be started from the program and their JSON results captured reliably (*Ansible Runner — Ansible Runner Documentation*, n.d.). To talk to different devices, the system uses libraries such as Netmiko and NAPALM for CLI and vendor-agnostic operations, ncclient for NETCONF when available, and ordinary HTTP/REST calls for SDN controllers (*Netmiko*, n.d., *Ncclient 0.6.9 Documentation*, n.d., *OpenDaylight Documentation*, n.d.). NetBox remains the single source of truth for inventory, and records are referenced by ID.

For testing and demonstrations, a repeatable lab setup is used. EVE-NG is the main simulator for campus topologies in this project. The implementation includes scripts to create, snapshot, and restore EVE-NG topologies. Then, Ansible runs against the virtual devices in check or dry-run mode (--check --diff) to produce diffs that are saved in the execution log. If a module does not fully support check mode, the system flags those tasks so the operator knows which steps need extra care.

For evidence retrieval and similarity search, the system can either pass small bundles directly to the LLM for immediate context or use an embeddings-based workflow with a vector database such as ChromaDB or Qdrant (*Chroma*, n.d.). The system also allows the use of alternatives such as pgvector or FAISS for future expansion. Embeddings can be created locally using tools like Sentence-Transformers or through hosted services, and the chosen

method is recorded to preserve provenance (*Sentence Transformers Documentation*, n.d.). The vector database holds small chunks of documents and links back to NetBox snapshots and telemetry samples so that the LLM can reference precise evidence (Song, 2025).

The frontend is designed to be simple and easy to use. Network topologies are shown using D3.js to make the connections and layout clear (Holtz, n.d.). The playbook editor uses Monaco Editor or Ace Editor so YAML and CLI commands are highlighted and checked for errors (*Monaco Editor*, n.d.). Live telemetry and execution logs are streamed in real time using WebSockets, allowing operators to follow playbook execution and quickly see verification results (*The WebSocket API (WebSockets) - Web APIs | MDN*, n.d.).

CogniShield is designed to handle operational concerns carefully. Important information such as intents, playbooks, execution logs, approvals, and audit records are stored in PostgreSQL. Schema changes are managed with Alembic to keep the database up to date (*Alembic 1.17.2 Documentation*, n.d.). Time-series telemetry data is stored in InfluxDB or Prometheus so it can be accessed quickly, and dashboards are built with Grafana for easy monitoring. Logs are written in a structured format like JSON and can be collected in Loki or an ELK stack for more detailed debugging (*Elastic Stack: (ELK) ElasticSearch, Kibana & Logstash*, n.d.). Secrets and credentials are protected. In the student lab, Ansible Vault is used for playbook secrets, and a local vault is recommended. For production, HashiCorp Vault is suggested for automatic secret rotation and stricter access control (*HashiCorp Developer*, n.d.).

The validator and executor are separate Python services to keep the system safe. The validator checks for syntax errors, vendor rules, and simple policies like reserved VLANs or IP conflicts. It then provides clear error or warning messages that the interface can show. The executor runs playbooks safely using ansible-runner. It enforces limits like rate control and serial execution for sensitive devices. Before making changes, it saves minimal snapshots and the exact configuration affected so that rollbacks can be done if something goes wrong.

Development, testing, and deployment follow common DevOps practices. The code is split into modules like API, adapters for LLMs, vector databases, NetBox, SDN controllers, the RAG builder, validator, executor, frontend, tests, and infrastructure files. This makes unit tests and integration tests easier. Continuous integration pipelines run linters, tests, image builds, and optional end-to-end checks. Docker Compose is used to start all the necessary services, including the database, telemetry, vector store, NetBox, backend, and frontend, so

the system can be tested locally. When possible, a small EVE-NG snapshot is used to simulate the full workflow safely.

Aside from that, hardware and deployment are kept simple for the lab. A standard server with 8–16 CPU cores, 32–64 GB of RAM, and SSD storage can run all services and the EVE-NG nodes for small topologies. If on-premise LLM inference is used later, a GPU is needed. For the student lab, hosted LLM APIs are recommended, and local inference is optional.

Finally, safety and governance are built into the system. Every LLM output is treated as a suggestion. It must go through template rules, static validation, dry-run testing, and human approval before any changes are applied to live devices (Mljourney, 2025). Audit information is stored for every action, including the prompt, the evidence used, the generated playbook, and operator decisions. This ensures that everything can be reviewed later. This careful approach makes CogniShield suitable for a student project while leaving room for future improvements and research.

3.3.4 Testing

The testing phase of CogniShield is designed to ensure that all components of the system operate safely and reliably before any configuration is applied to a network device. Because the platform integrates LLM-generated playbooks, automated validation, and real device interactions, the testing strategy adopts a layered and systematic approach. This helps prevent unexpected behaviour, reduces operational risks, and ensures that the system behaves predictably throughout its workflow (Stojanov, 2025).

The first layer of testing focuses on **unit testing**, which verifies the correctness of individual modules in isolation. Using tools such as *pytest*, each connector (e.g., NetBox, SDN controller, vector database), RAG retrieval function, and prompt template handler is tested with mocked inputs. This ensures consistency and prevents external dependencies from affecting the results. Unit tests also check for proper error handling, such as malformed intents, missing inventory data or unexpected LLM responses (Iuliia Volkova, 2020). Python code quality checks are performed using linters, while generated Ansible fragments are validated with **ansible-lint** to ensure that syntax issues or unsafe patterns are detected early.

The next layer is **integration testing**, which examines how different subsystems behave when combined. A lightweight test environment is deployed using Docker Compose, including PostgreSQL, NetBox, a basic SDN controller (or simulated API), a time-series store and an EVE-NG lab environment. Through this setup, the system is tested on realistic tasks such as VLAN configuration, ACL updates, interface shutdown operations and basic remediation triggers. Integration tests verify that the retrieval pipeline, grounding logic, LLM adaptor response, static validators and dry-run executor can operate together smoothly. Negative testing is also included, where invalid or conflicting intents are intentionally introduced to confirm that the platform correctly rejects unsafe requests.

Building on integration validation, **system and end-to-end (E2E) testing** focuses on the complete workflow from intent submission all the way to final post-execution verification (Testim, 2025). In this stage, EVE-NG plays a central role by simulating a small campus network environment with routers, switches and realistic topologies. An E2E scenario typically starts with an operator submitting an intent, followed by context retrieval, LLM-based playbook generation, static validation and a dry-run executed via Ansible check mode. Once reviewed, the playbook is applied to EVE-NG devices using the orchestrator.

After execution, the system performs post-checks to verify whether the final device state matches the expected configuration. Rollback behaviour is also tested by deliberately causing verification failures and ensuring that the system can revert devices to their previous configuration. These tests ensure that every subsystem works together correctly in realistic operational conditions, and that safety mechanisms function as intended.

3.4 Technology Deployed and Hardware

The implementation of CogniShield is designed for reproducibility in a university lab while remaining modular and extensible for later research work. The primary implementation language is Python (targeting a modern release such as Python 3.13.x) because of its rich networking ecosystem, strong support for automation tools and extensive ML/LLM integrations. The overall system is implemented as a service-based stack: a backend API and orchestration layer written in Python, a lightweight single-page frontend in React, and several specialized auxiliary services (time-series datastore, vector store, telemetry exporters, and an automation runner). All components are containerised so the platform can be run on a single lab host with Docker Compose and scaled to Kubernetes in future.

3.4.1 Languages, Frameworks and Core Libraries

- **Backend:** Python 3.13.x using FastAPI (preferred for async endpoints and WebSocket support) or Flask if a simpler stack is required. Use Uvicorn/Gunicorn for ASGI/WGI workers.
- **Frontend:** React (create-react-app or Vite) as a single-page application. Useful UI libraries: Tailwind CSS for rapid styling, Visual Studio Code for YAML/CLI editing, and a lightweight component library (e.g., DaisyUI, ShadCN UI or Material UI) for consistent elements.
- **Visualization:** D3.js for custom visual overlays (bandwidth/metric sparklines, time-based charts) and Cytoscape.js or vis.js for topology graphs (planned vs live and topology diffs).
- **LLM integration:** Adapters for both hosted and local models. Support for:
    - **Gemini API** (Google Gemini Open API) and other hosted providers (OpenAI, Anthropic) via pluggable adapters.
    - Local model support via Hugging Face Transformers or quantized runtimes (ggml/llama.cpp) if experimental on-prem inference is attempted.
- **RAG / embeddings / Vector DBs:** support for ChromaDB and Qdrant (and fallback support in docs for pgvector or FAISS). Embedding libraries include

Sentence-Transformers (local) and hosted embedding endpoints (OpenAI / Google Embeddings).

- **Automation & Execution:** Ansible Core (community edition) as the execution engine; use the **ansible-runner** Python library to drive playbooks programmatically and to collect structured results. Use Ansible check mode and --diff for dry-run.
- **Network device connectivity:** Netmiko and NAPALM for CLI and multi-vendor support; pynetbox for NetBox integration; ncclient for NETCONF; requests/aiohttp for REST/SDN northbound APIs.
- **Telemetry & Monitoring:** Prometheus or InfluxDB for metrics; Grafana for visual dashboards; Telegraf/SNMP exporters or gNMI exporters to collect telemetry from simulated or lab devices.
- **Storage & metadata:** PostgreSQL (relational metadata store); use Alembic for schema migrations.
- **Logging & observability:** Loki or ELK (Elasticsearch/Fluentd/Kibana) for log aggregation; structured JSON logs from the backend.
- **Secrets & credential management:** HashiCorp Vault for production-grade secrets. For the student lab, Ansible Vault and environment-variable based secrets are acceptable with strong procedural controls.
- **Virtualization environment:** Proxmox is used to create multiple isolated virtual machines or containers to host the CogniShield components. This enables reproducible deployments, resource management, snapshotting, and easy rollback of experimental configurations, ensuring that students can test safely without impacting other virtualized services.
- **Network lab simulation:** EVE-NG is used to simulate campus network environments, including devices, links, and topologies. This allows students to validate playbooks, test automated remediation, and safely observe the effect of configuration changes on virtual networks. EVE-NG works in conjunction with the Proxmox virtual machines, providing a realistic and flexible testbed for the full CogniShield system.

3.4.2 Hardware and Deployment Considerations

- **Student lab (MVP):** A modest server with 8–16 vCPUs, 32–64 GB RAM, and SSD storage is enough to run the Docker Compose stack and small LLM inference if using CPU-quantized models. EVE-NG requires additional RAM for network images depending on topology size.

- **Optional GPU inference:** If experimenting with large models locally, a single GPU (e.g., NVIDIA RTX 30xx/40xx) with appropriate CUDA/driver support will be required (Gangireddy Bala Akshay, 2025). For most FYP purposes, using hosted LLM APIs (Gemini, OpenAI) is simpler and cheaper in time.

3.4.3 Security, Privacy and Safe-By-Design Choices

- **Least privilege:** Device accounts used in the lab have minimal rights while production suggestions are documented but out of scope.

- **LLM safety:** Enforce template-controlled outputs, attach RAG evidence to every generation and require static validation and dry-run before apply. Always ensure that the operator remains involved in the process by default.

- **Secrets:** Use HashiCorp Vault in realistic setups and Ansible Vault for the demo lab. Access to Vault is role-restricted and logged.

3.5 SDG Alignment

Based on the benefits CogniShield could bring to the world, this project is believed to be highly aligning with SDG 9, which is Industry, Innovation, and Infrastructure, as CogniShield promotes innovation in the industry field of network and security administration through the integration of artificial intelligence, automation, and open-source collaboration. By making the platform open-source and accessible to everyone, CogniShield could provide an equal and fair opportunity that helps students, researchers, and educators learn about AI-driven automation, cybersecurity, and network engineering in a hands-on manner. In the long run, these benefited individuals can also innovate on this platform and provide more advanced or customized services to society and enterprise, thereby promoting a community-driven development. Last but not least, the reason why CogniShield is in favor of SDG 9 is because the platform could provide startups and SMEs with a network and security tool that can increase their configuration workflow productivity, reduce manual workload as well as enhance the security posture, thereby improving the overall efficiency with the help of intelligent automation.

3.6 Conclusion & Future Works

In conclusion, this chapter has presented a comprehensive and practical Software Requirements Design (SRD) for CogniShield, with careful consideration given to both innovative features and operational safety. Since the system aims to integrate LLM-driven automation into a campus networking environment, it is important to establish a solid foundation before any development work begins. By adopting the Waterfall SDLC with seven well-defined stages, the project can follow a structured and systematic approach, which is especially helpful when dealing with network configurations where unexpected errors may cause service disruptions.

Architecture and Implementation Guidelines

The architecture described in this chapter emphasises modular design, the combination of LLM and retrieval techniques, and the importance of maintaining human oversight. These elements help reduce uncertainty and promote clearer explainability, which is crucial when AI-generated suggestions influence real operational decisions. In addition, the implementation guidelines such as recommended programming languages, directory organisation, containerisation strategies, and software versions provide a practical reference point for developers to start building the system in a consistent and organised manner.

Testing Strategy and Hardware Recommendations

The testing plan is also designed in a thorough and layered way. It helps ensure that functional issues, safety concerns, and performance limitations can be identified at different stages through unit, integration, and end-to-end testing. This is particularly important because generative models can behave unpredictably if not properly validated. The inclusion of hardware recommendations further supports the overall development process, showing a realistic path from initial student lab experiments to a smaller-scale pilot deployment in an actual campus setting.

Future Works and Project Progression

With these requirements and design considerations properly documented, the next steps of the project become clearer. The project can be then begun by preparing detailed design artefacts, including domain-driven design documents, API specifications, and structured test matrices. After that, the development of initial prototypes can take place to verify the system's assumptions and ensure that the architecture behaves as intended. From there, pilot deployments can be introduced gradually, allowing the system's features to be expanded step by step while still maintaining safety, auditability, and operational control. By progressing in this careful and responsible manner, CogniShield has the potential to grow into a dependable platform that demonstrates both technical capability and a disciplined approach to using AI in network automation.

References

5 Intent-Based networking vendors. (n.d.). NetworkComputing. https://www.networkcomputing.com/network-infrastructure/5-intent-based-networking-vendors

About Grafana | Grafana documentation. (n.d.). Grafana Labs. https://grafana.com/docs/grafana/latest/introduction/

Access, distribution, and core layers explained. (2025, July 23). ComputerNetworkingNotes. https://www.computernetworkingnotes.com/ccna-study-guide/access-distribution-and-core-layers-explained.html#:~:text=functions%20in%20detail.-,The%20access%20layer,that%20contains%20two%20access%20switches.

Ansible Documentation — Ansible Community Documentation. (n.d.). https://docs.ansible.com/ansible/latest/index.html

Ansible Runner — Ansible Runner Documentation. (n.d.). https://ansible-runner.readthedocs.io/en/latest/

Bender, E. M., Gebru, T., McMillan-Major, A., & Shmitchell, S. (2021). *On the dangers of stochastic parrots: Can language models be too big?* Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency. https://doi.org/10.1145/3442188.3445922

Carnegie Mellon Software Engineering Institute. (2021). *Benefits and challenges of SOAR platforms*. SEI Blog. https://www.sei.cmu.edu/blog/benefits-and-challenges-of-soar-platforms/

Chauhan, V., & Chauhan, V. (2025, October 1). What is Prompt Injection Attack? (Types, Risks & Prevention). *Prismetric*. https://www.prismetric.com/prompt-injection/?utm_source=chatgpt.com

Chroma. (n.d.). Chroma. https://www.trychroma.com/docs/

Cisco DNA Center Reviews & Product Details. (n.d.). G2.com. https://www.g2.com/products/cisco-cisco-dna-center/reviews

Cisco introduces AI-First Approach to IT Operations -- Campus Technology. (2025, June 20). Campus Technology. https://campustechnology.com/articles/2025/06/20/cisco-introduces-ai-first-approach-to-it-operations.aspx#:~:text=Cisco%20AI%20Assistant:%20A%20conversational,%2C%20enhancing%20decision%2Dmaking%20processes.

Cisco Systems. (2008). *Enterprise campus architecture overview* (Campus design guide). Cisco Systems. https://www.cisco.com/c/en/us/td/docs/solutions/Enterprise/Campus/campover.html

Cisco Systems. (2019). *Campus LAN and wireless LAN solution design guide*. Cisco Systems. https://www.cisco.com/c/en/us/td/docs/solutions/CVD/Campus/cisco-campus-lan-wlan-design-guide.html

Cisco Systems. (2020). *Intent-Based Networking and Extending the Enterprise* (white paper). Cisco Systems.

https://www.cisco.com/c/en/us/solutions/collateral/internet-of-things/nb-09-intent-based-iot-wp-cte-en.pdf

Cisco Systems. (2023). *Catalyst Center / DNA Center product information and licensing* (data sheet/product brief). Cisco Systems. https://www.cisco.com/c/en/us/products/collateral/cloud-systems-management/dna-center/nb-06-dna-center-data-sheet-cte-en.html

Conry-Murray, D. (2025, February 27). *Juniper brings Mist AI to routing and opens door to Third-Party LLMs*. Packet Pushers Interactive LLC. https://packetpushers.net/blog/juniper-brings-mist-ai-to-routing-and-opens-door-to-third-party-llms/

Contributor, T. (2025, July 11). *What is a campus network?* Search Networking. https://www.techtarget.com/searchnetworking/definition/campus-network

Elastic Stack: (ELK) ElasticSearch, Kibana & Logstash. (n.d.). Elastic. https://www.elastic.co/what-is/elk-stack

EVE-NG Ltd. (2025, October 1). *Home* -. https://www.eve-ng.net/

FastAPI. (n.d.). https://fastapi.tiangolo.com/

Flores, D. (2021, March 29). *Network Automation 101 - Skills and technologies*. Packet Coders - Learn Network Automation. https://www.packetcoders.io/network-automation-101-skills-and-technologies/?utm_source=chatgpt.com

Free and open source software in education | Research Starters | EBSCO Research. (n.d.). EBSCO. https://www.ebsco.com/research-starters/computer-science/free-and-open-source-software-education?utm_source=chatgpt.com

Gangireddy Bala Akshay. (2025, August 16). *Beginner's guide: Setting up Llama.CPP for local LLM experiments (GPU optimized)*. https://balaakshay.medium.com/beginners-guide-setting-up-llama-cpp-for-local-llm-experiments-gpu-optimized-291adc5b7ba2

Gazala, S. (2025, November 19). *Waterfall Model in SDLC: Its phases, advantages, and applications*. https://www.lambdatest.com/learning-hub/waterfall-model?utm_source=chatgpt.com

GeeksforGeeks. (2025, September 30). *Waterfall Model Software Engineering*. GeeksforGeeks. https://www.geeksforgeeks.org/software-engineering/waterfall-model/

GeeksforGeeks. (2025a, July 15). *NonFunctional requirements in software engineering*. GeeksforGeeks. https://www.geeksforgeeks.org/software-engineering/non-functional-requirements-in-software-engineering/?utm_source=chatgpt.com

GeeksforGeeks. (2025a, July 23). *What is Agile Methodology?* GeeksforGeeks. https://www.geeksforgeeks.org/software-testing/what-is-agile-methodology/

GeeksforGeeks. (2025c, October 18). *Functional and non functional requirements*. GeeksforGeeks. https://www.geeksforgeeks.org/software-engineering/functional-vs-non-functional-requirements/?utm_source=chatgpt.com

Ggml-Org. (n.d.). *GitHub - ggml-org/llama.cpp: LLM inference in C/C++*. GitHub. https://github.com/ggerganov/llama.cpp

Grafana Labs. (n.d.). *Get started with Grafana and Prometheus*. https://grafana.com/docs/grafana/latest/getting-started/get-started-grafana-prometheus/

Hickey, N. (2024, November 13). *AI-Powered Network Troubleshooting with NetBrain: Faster, Smarter, and Proactive*. Packet Pushers. https://packetpushers.net/blog/ai-powered-network-troubleshooting-with-netbrain-faster-smarter-and-proactive/#:~:text=NetBrain's%20AI%20Co%2DPilot%20allows,configuration%20issues%20affecting%20the%20region.

Holtz, Y. (n.d.). *Network Graph | The D3 Graph Gallery*. https://d3-graph-gallery.com/network

Huang, L., et al. (2023). *A survey on hallucination in large language models* (arXiv preprint). https://arxiv.org/abs/2311.05232

Huawei Technologies Co., Ltd. (2025). *Introduction to the Intelligent Simplified Campus Network Solution*. Huawei Configuration Guide. https://support.huawei.com/enterprise/en/doc/EDOC1100365116/8e2de9c0/introduction-to-the-intelligent-simplified-campus-network-solution

Huawei Technologies Co., Ltd. (n.d.). *Underlay Network Architecture Design*. https://info.support.huawei.com/hedex/api/pages/EDOC1100331202/FEM1012J/06/resources/en-us_topic_0268148565.html#:~:text=Core%20layer,-The%20core%20layer&text=It%20connects%20to%20various%20components,with%20more%20than%20three%20departments.

IMaster NCE Campus — Huawei Enterprise. (n.d.). Huawei Enterprise. https://e.huawei.com/en/products/network-analysis/imaster-nce-campus

index | NetBox Documentation. (2025, May 30). NetBox Labs Documentation. https://netboxlabs.com/docs/netbox/

InfluxData. (2021, December 10). *InfluxDB: Open Source Time Series Database | InfluxData*. https://www.influxdata.com/solutions/network-monitoring/?utm_source=chatgpt.com

InfluxDB OSS v2 Documentation. (n.d.). InfluxData Inc. https://docs.influxdata.com/influxdb/

Ismail, R., Kurnia, R., Brata, Z. A., Nelistiani, G. A., Heo, S., Kim, H., & Kim, H. (2025). Toward robust security orchestration and automated response in security operations centers with a hyper-automation approach using agentic AI. *Information, 16*(5), 365.

https://www.mdpi.com/2078-2489/16/5/365

Iuliia Volkova (2020, June 1). *Integration testing for bunch of services with Pytest & Docker compose*. https://xnuinside.medium.com/integration-testing-for-bunch-of-services-with-pytest-docker-compose-4892668f9cba

Juniper Networks. (n.d.). *May 18th 2023 Updates | MIST | Documentation | Juniper Networks*. https://www.juniper.net/documentation/us/en/software/mist/product-updates/2023/may-18th-2023-updates.html

Juniper Networks. (n.d.). *Understanding the NETCONF XML Management Protocol | Junos OS | Juniper Networks*. https://www.juniper.net/documentation/us/en/software/junos/netconf/topics/topic-map/netconf-overview.html

Juniper Networks. (n.d.-a). *Marvis AI*. https://www.juniper.net/us/en/products/marvis-ai.html#:~:text=Is%20natural%20language%20processing%20(NLP,regarding%20documentation%20and%20support%20issues.

Landau, P. (2025, June 23). *WBS and Gantt Chart: How to Use This Project Management Duo*. ProjectManager. https://www.projectmanager.com/blog/wbs-and-gantt-chart

Letaw, L. (2024, January 1). *Requirements*. Handbook of Software Engineering Methods. https://open.oregonstate.education/setextbook/chapter/requirements/?utm_source=chatgpt.com

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). *Retrieval-augmented generation for knowledge-intensive NLP tasks*. NeurIPS 2020.

https://arxiv.org/abs/2005.11401

Malsam, W. (2025, August 2). *Project Planning Phase: key activities and outcomes*. ProjectManager. https://www.projectmanager.com/blog/project-planning-phase?utm_source=chatgpt.com

Mljourney. (2025, November 15). *LLM audit and Compliance Best Practices*. ML Journey. https://mljourney.com/llm-audit-and-compliance-best-practices/?utm_source=chatgpt.com

Monaco Editor. (n.d.). https://microsoft.github.io/monaco-editor/?utm_source=chatgpt.com

Netbox-Community. (n.d.). *GitHub - netbox-community/pynetbox: Python API client library for Netbox.* GitHub. https://github.com/netbox-community/pynetbox

NetBox Project. (n.d.). *NetBox documentation — source of truth for network infrastructure*. https://netbox.readthedocs.io/

NetBrain. (2024, December 17). *Network Automation | Solutions | NetBrain Technologies*. NetBrain. https://www.netbraintech.com/solution/network-automation/

netmiko. (n.d.). Netmiko. https://ktbyers.github.io/netmiko/

Network Computing. (2016). *Campus network design models*. https://www.networkcomputing.com/data-center-networking/campus-network-design-models

OECD. (2021). *The digital transformation of SMEs*. OECD Publishing. https://www.oecd.org/industry/smes/The-digital-transformation-of-SMEs.pdf

Open Networking alternative to Cisco, Juniper, Arista, Nokia. (2025, May 21). IP Infusion. https://www.ipinfusion.com/open-networking-alternative-to-cisco-juniper-arista-nokia-and-huawei-switches-and-routers/

Overview | Prometheus. (n.d.). https://prometheus.io/docs/introduction/overview/

OWASP Top 10 for LLM & Generative AI Security. (2025, April 17). *LLM01:2025 Prompt Injection - OWASP Gen AI Security Project*. OWASP Gen AI Security Project. https://genai.owasp.org/llmrisk/llm01-prompt-injection/

ProductPlan. (2025, July 22). *What is MoSCoW Prioritization? | Overview of the MoSCoW Method*. https://www.productplan.com/glossary/moscow-prioritization/?utm_source=chatgpt.com

Prometheus. (n.d.). *Overview — Prometheus monitoring system*. https://prometheus.io/docs/introduction/overview/

Proxmox. (n.d.). *Proxmox Virtual Environment*. https://www.proxmox.com/en/products/proxmox-virtual-environment/overview

Quick start – react. (n.d.). https://reactjs.org/docs/getting-started.html

Ramlochan, S. (2025, April 6). *Improving Large Language Models with Retrieval Augmented Generation*. Prompt Engineering Institute. https://promptengineering.org/improving-large-language-models-with-retrieval-augmented-generation/?utm_source=chatgpt.com

Requirements Traceability Matrix: Definition, benefits, and Examples | Perforce Software. (n.d.). https://www.perforce.com/resources/alm/requirements-traceability-matrix?utm_source=chatgpt.com

SentenceTransformers Documentation — Sentence Transformers documentation. (n.d.-b). https://www.sbert.net/

Software-Defined Networking (SDN) Definition - Open Networking Foundation. (2020, June 3). Open Networking Foundation. https://opennetworking.org/sdn-definition/

Song, P. (2025, July 4). *What is a Vector Database for LLM: Complete Guide*. ML Journey. https://mljourney.com/what-is-a-vector-database-for-llm-complete-guide/

Steven. (2025, July 30). *LLM Security Guide 2025: Prevent prompt injection and data leakage in production*. System Debug. https://sysdebug.com/posts/llm-security-prompt-injection-data-leakage/

Stojanov, D. (2025, August 1). Understanding the layers of testing architecture. *TestDevLab*. https://www.testdevlab.com/blog/what-are-the-layers-of-testing-architecture/?utm_source=chatgpt.com

Testim. (2025, March 14). What is end to end testing? A helpful introductory guide. *AI-driven E2E automation with code-like flexibility for your most resilient tests*. https://www.testim.io/blog/end-to-end-testing-guide/?utm_source=chatgpt.com

The Python Standard Library. (n.d.). Python Documentation. https://docs.python.org/3.14/library/index.html

The WebSocket API (WebSockets) - Web APIs | MDN. (n.d.). https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API?utm_source=chatgpt.com

Towards Trustworthy AI: Characterizing User-Reported Risks across LLMs "In the Wild." (n.d.). https://arxiv.org/html/2509.08912v1

Transformers. (n.d.). https://huggingface.co/docs/transformers/index

Transforming Network Configuration Management: Challenges and Solutions. (2022). In *https://backbox.com/*. https://backbox.com/wp-content/uploads/TransformingNCM_ChallengesAndSolutions_Whitepaper_2024.pdf?utm_source=chatgpt.com

TutorialsCampus. (n.d.). *SDLC Waterfall model*. TutorialsCampus. https://www.tutorialscampus.com/sdlc/waterfall-model.htm

U.S. International Trade Administration. (2023). *U.S. SME access and use of digital tools* (report). https://www.trade.gov/sites/default/files/2023-06/SME_Digital_Tools.pdf

Understanding Cisco DNA and Catalyst Software eBook. (2024, August 9). Cisco. https://www.cisco.com/c/en/us/products/collateral/software/dna-software-ebook-cte.html?utm_source=chatgpt.com

Upravnik. (2022, December 18). *Cisco three-layer hierarchical model*. Study CCNA. https://study-ccna.com/cisco-three-layer-hierarchical-model/

Vault product documentation | Vault | HashiCorp Developer. (n.d.). Vault Product Documentation | Vault | HashiCorp Developer. https://www.vaultproject.io/docs

Wang, C., Scazzariello, M., Farshin, A., Ferlin, S., Kostić, D., & Chiesa, M. (2024). *NetConfEval: Can LLMs Facilitate Network Configuration?* ACM. https://dl.acm.org/doi/10.1145/3656296

Wang, C., Scazzariello, M., Farshin, A., Ferlin, S., Kostić, D., & Chiesa, M. (2024). *NetConfEval: Can LLMs facilitate network configuration?* Proceedings of ACM CoNEXT 2024. https://dejankosticgithub.github.io/documents/publications/netconfeval-conext24.pdf

Watch a playbook overview. (n.d.). [Video]. Splunk. https://www.splunk.com/en_us/products/splunk-security-orchestration-and-automation.html

Welcome to Alembic's documentation! — Alembic 1.17.2 documentation. (n.d.). https://alembic.sqlalchemy.org/en/latest/

Welcome to the OpenDaylight Handbook! — OpenDaylight Documentation Oxygen documentation. (n.d.). https://docs.opendaylight.org/en/stable-oxygen

What is Unified Modeling Language (UML)? (n.d.). https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-uml/?utm_source=chatgpt.com

Xavier_Elliot. (2025, June 5). Intent-Based network automation in modern infrastructure. *ACE Journal*. https://www.acejournal.org/2025/06/05/intent-based-network-automation?utm_source=chatgpt.com

"What is Docker?" (2024, September 10). Docker Documentation. https://docs.docker.com/get-started/docker-overview